COMSATS UNIVERSITY ISLAMABAD

ABBOTTABAD CAMPUS, PAKISTAN

CYBERSENTINEL AI – INTELLIGENT CYBERSECURITY TOOLKIT FOR PHISHING, MALWARE, AND THREAT DETECTION

BY

MOHAVIA ARIF
CIIT/FA22-BCS-084/ATD

ANAS BASHIR
CIIT/FA22-BCS-081/ATD

ABDUL SAMAD PARACHA
CIIT/FA22-BCS-056/ATD

SUPERVISOR

DR. MAZHAR ALI

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (2022–2026)

COMSATS UNIVERSITY ISLAMABAD
ABBOTTABAD CAMPUS, PAKISTAN

CYBERSENTINEL AI – INTELLIGENT CYBERSECURITY TOOLKIT FOR PHISHING, MALWARE, AND THREAT DETECTION

A PROJECT PRESENTED TO
COMSATS UNIVERSITY ISLAMABAD, ABBOTTABAD CAMPUS

IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF

BACHELOR OF SCIENCE IN COMPUTER SCIENCE (2022–2026)

BY

MOHAVIA ARIF
CIIT/FA22-BCS-084/ATD

ANAS BASHIR
CIIT/FA22-BCS-081/ATD

ABDUL SAMAD PARACHA
CIIT/FA22-BCS-056/ATD

DECLARATION

We hereby declare that this project titled “CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection” is the result of our own efforts and has not been copied from any external source. All sources of information used in this project have been properly acknowledged and referenced.

We further declare that this project report has not been submitted, either in whole or in part, for any other degree or qualification at this or any other institution.

If any part of this work is found to be plagiarized or reproduced from another source without proper citation, we accept full responsibility and the consequences as determined by the university.

MOHAVIA ARIF	**ANAS BASHIR**	**ABDUL SAMAD PARACHA**
----------	----------	----------

CERTIFICATE OF APPROVAL

It is to certify that the Final Year Project titled **"CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection"** has been developed by **Mohavia Arif (CIIT/FA22-BCS-084/ATD)**, **Anas Bashir (CIIT/FA22-BCS-081/ATD)**, and **Abdul Samad Paracha (CIIT/FA22-BCS-056/ATD)** under the supervision of **Dr. Mazhar Ali**.

In our opinion, this project is fully adequate in scope and quality for the award of the degree of **Bachelor of Science in Computer Science (2022–2026)**.

Supervisor

External Examiner

Head of Department
(Department of Computer Science)

EXECUTIVE SUMMARY

In today's rapidly evolving digital landscape, cybersecurity threats such as phishing attacks, malicious URLs, and malware infections have become increasingly common, particularly affecting individuals and small businesses with limited access to advanced security tools. In Pakistan, the lack of affordable, user-friendly, and localized cybersecurity solutions further increases the vulnerability of users to online scams, data breaches, and financial fraud.

To address these challenges, CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection is developed as a web-based platform that provides real-time threat detection using Artificial Intelligence (AI) and Machine Learning (ML). The system integrates multiple detection modules into a unified interface, enabling users to analyze suspicious emails, URLs, and files through a simple and intuitive dashboard.

The phishing detection module utilizes Natural Language Processing (NLP) techniques, including TF-IDF vectorization and a trained Logistic Regression model, to classify email content as safe, suspicious, or phishing based on linguistic patterns and contextual features. The malicious URL analyzer employs machine learning algorithms and external threat intelligence sources such as PhishTank and AbuseIPDB to evaluate domain credibility and detect harmful links. Additionally, the malware detection module integrates with the VirusTotal API to analyze uploaded files and identify known malware signatures using hash-based scanning.

The system is built using a modern technology stack consisting of a React-based frontend for user interaction, a Flask backend for API handling, and machine learning libraries such as scikit-learn for model implementation. The modular architecture ensures scalability, allowing future integration of advanced features such as threat intelligence dashboards, browser extensions, and mobile applications.

CyberSentinel AI provides results in a clear and user-friendly manner using color-coded risk indicators (green for safe, yellow for suspicious, and red for malicious), along with confidence scores and actionable recommendations. The platform also supports report generation and logging mechanisms to enhance usability and auditability.

Overall, CyberSentinel AI offers an accessible, efficient, and intelligent cybersecurity solution tailored to the needs of non-technical users. By combining AI-driven analysis with a simplified interface, the system contributes to improving cybersecurity awareness and reducing the risk of digital threats in Pakistan's growing online environment.

ACKNOWLEDGEMENT

All praise is to Almighty Allah, whose countless blessings, guidance, and wisdom enabled us to successfully complete this project.

We would like to express our sincere gratitude to our respected supervisor, **Dr. Mazhar Ali**, for his continuous support, valuable guidance, and constructive feedback throughout the development of this project. His encouragement and insightful suggestions played a vital role in shaping the direction and quality of our work.

We are also thankful to the faculty members of the Department of Computer Science, COMSATS University Islamabad, Abbottabad Campus, for providing us with the knowledge, resources, and academic environment necessary for completing this project successfully.

Furthermore, we extend our heartfelt appreciation to our parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength during our academic journey.

Finally, we would like to acknowledge everyone who directly or indirectly contributed to the completion of this project.

MOHAVIA ARIF **ANAS BASHIR** **ABDUL SAMAD PARACHA**

---------- ---------- ----------

ABBREVIATIONS

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSV	Comma Separated Values
DB	Database
FR	Functional Requirement
ML	Machine Learning
NLP	Natural Language Processing
SVM	Support Vector Machine
TF-IDF	Term Frequency – Inverse Document Frequency
UI	User Interface
URL	Uniform Resource Locator
VT	VirusTotal

Table of contents

Table of Figures

Table of Tables

1. INTRODUCTION

This chapter presents an overview of the project titled **"CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection."** It introduces the purpose, significance, and background of the system, along with its relevance to academic coursework. The chapter also includes a review of existing systems, analysis of their limitations, and the methodology adopted for developing the proposed solution.

1.1 Brief

CyberSentinel AI is a web-based cybersecurity platform designed to detect and analyze digital threats in real time using Artificial Intelligence (AI) and Machine Learning (ML) techniques. The system focuses on three primary threat categories: phishing emails, malicious URLs, and malware-infected files.

The platform enables users to submit suspicious content—such as email text, web links, or files—and receive instant classification results indicating whether the input is safe, suspicious, or malicious. The system presents results using color-coded indicators along with confidence scores, making it easy for non-technical users to understand potential risks.

The core functionality of the system is based on an NLP-driven phishing detection module, which utilizes TF-IDF vectorization and a Logistic Regression model to analyze textual patterns in emails. Additional modules for URL analysis and malware detection are integrated using machine learning techniques and external threat intelligence APIs such as PhishTank and VirusTotal.

The primary objective of CyberSentinel AI is to provide an accessible, affordable, and user-friendly cybersecurity solution tailored for individuals and small organizations, particularly in regions where advanced security tools are not readily available.

1.2 Relevance to Course Modules

The development of CyberSentinel AI integrates knowledge and concepts from multiple courses studied during the Bachelor of Science in Computer Science program. It demonstrates the practical application of theoretical concepts in a real-world cybersecurity solution.

Key course contributions include:

• **Machine Learning:** Implementation of classification models such as Logistic Regression for phishing detection and feature-based models for URL analysis.

• **Natural Language Processing (NLP):** Text preprocessing, tokenization, and TF-IDF vectorization for analyzing email content.

• **Software Engineering:** Application of Software Development Life Cycle (SDLC), modular system design, and Agile development methodology.

• **Web Development:** Development of a responsive frontend using React.js and backend API integration using Flask.

• **Database Systems:** Management of user data, scan history, and logs using structured storage systems.

• **Computer Networks & Security:** Understanding of cyber threats such as phishing, malware, and malicious URLs, and techniques for detecting and mitigating them.

This project effectively bridges academic learning with practical implementation, providing hands-on experience in designing and deploying an intelligent cybersecurity system.

1.3 Project Background

With the rapid growth of internet usage and digital services, cybersecurity threats have become increasingly prevalent. Users frequently encounter phishing emails, fraudulent websites, and malicious software designed to steal sensitive information or compromise systems.

In Pakistan, many individuals and small businesses lack access to reliable cybersecurity tools due to high costs, technical complexity, and limited awareness. As a result, users often rely on guesswork when dealing with suspicious content, leading to increased vulnerability to cyberattacks.

Existing solutions such as antivirus software and online scanning tools are often either too complex for non-technical users or require paid subscriptions. Additionally, many systems focus on a single type of threat and do not provide a unified platform for analyzing multiple types of cybersecurity risks.

CyberSentinel AI is developed to address these challenges by offering a simple, web-based platform that combines phishing detection, URL analysis, and malware scanning into a single system. By leveraging AI and ML techniques, the system provides real-time threat detection and enhances user awareness, making cybersecurity more accessible and effective for a broader audience.

1.4 Literature Review

Several existing systems and platforms have been developed to address cybersecurity threats such as phishing, malicious URLs, and malware detection. These systems provide valuable insights into threat detection techniques but also exhibit certain limitations.

One of the widely used platforms is VirusTotal, which allows users to scan files and URLs using multiple antivirus engines. While it is effective in identifying known threats, it is primarily designed for technical users and lacks Natural Language Processing (NLP) capabilities for detecting phishing emails. Additionally, the interpretation of its results often requires technical expertise.

Another notable system is the Kaspersky Threat Intelligence Portal, which provides advanced threat analysis and malware detection services. However, it is mainly targeted toward enterprise environments and requires paid subscriptions, making it less accessible for individual users and small businesses.

Cisco Talos Intelligence is another cybersecurity platform that offers comprehensive threat intelligence and network security insights. Despite its effectiveness, it is complex and not designed for non-technical users, limiting its usability for the general public.

In the academic domain, several AI-based phishing detection systems have been proposed using Machine Learning and Deep Learning techniques such as Support Vector Machines (SVM), Logistic Regression, and BERT-based models. These systems demonstrate high accuracy in controlled environments but often lack real-world deployment, integration with external APIs, and user-friendly interfaces.

Overall, while existing systems provide effective threat detection, they often lack accessibility, integration of multiple threat detection methods, and simplicity required for non-technical users.

1.5 Analysis from Literature Review

The analysis of existing systems reveals several key limitations that highlight the need for an improved cybersecurity solution. Most platforms focus on a single type of threat, such as file scanning or URL analysis, and do not provide a unified system for detecting multiple threats simultaneously.

Additionally, many existing tools are either too complex for non-technical users or require paid subscriptions, limiting their accessibility. The lack of user-friendly interfaces and clear result interpretation further reduces their effectiveness for everyday users.

Another major limitation is the reliance on static databases and rule-based detection methods, which are not effective against evolving cyber threats. Many systems also fail to provide real-time analysis or do not integrate multiple data sources for improved accuracy.

CyberSentinel AI addresses these limitations by integrating phishing detection, URL analysis, and malware scanning into a single platform. It utilizes AI and ML techniques for adaptive threat detection and combines them with external threat intelligence APIs to provide up-to-date and accurate results.

Furthermore, the system emphasizes simplicity and accessibility by offering a user-friendly web interface with clear visual indicators and actionable insights. This makes it suitable for non-technical users while maintaining effectiveness in threat detection.

1.6 Methodology and Software Process Model

The development of CyberSentinel AI follows an Object-Oriented Methodology (OOM) combined with an Agile Software Development Life Cycle (SDLC). This approach enables modular development, flexibility, and continuous improvement throughout the project lifecycle.

The Object-Oriented Methodology is used to structure the system into independent modules such as phishing detection, URL analysis, malware scanning, and user management. Each module is designed as a separate component, ensuring scalability, maintainability, and ease of integration.

The Agile methodology is adopted to manage the development process through iterative and incremental cycles. The project is divided into multiple phases, including dataset preparation, model development, backend implementation, frontend integration, and testing. Each phase is developed and refined through continuous feedback and evaluation.

1.6.1 Rationale for Selected Methodology

The Object-Oriented approach is selected because it supports modular system design, allowing different components of the system to be developed and tested independently. It also enhances code reusability and simplifies system maintenance.

The Agile development model is chosen due to its flexibility and suitability for AI-based systems, where continuous refinement and testing are required. It allows the team to adapt to changes, improve model performance, and integrate new features efficiently.

Together, these methodologies ensure that CyberSentinel AI is developed as a scalable, efficient, and well-structured cybersecurity platform capable of evolving with future requirements.

2. Problem Definition

This chapter defines the core problem addressed by CyberSentinel AI and explains the limitations of current cybersecurity practices. It also outlines the deliverables of the proposed system and provides an overview of existing solutions.

2.1 Problem Statement

The increasing reliance on digital communication and online platforms has led to a significant rise in cybersecurity threats such as phishing attacks, malicious URLs, and malware infections. Users are frequently exposed to deceptive emails, fraudulent websites, and harmful files that are designed to steal sensitive information, including login credentials, financial data, and personal details.

In Pakistan, individuals and small businesses are particularly vulnerable due to limited cybersecurity awareness and lack of access to affordable and easy-to-use security tools. Many users are unable to distinguish between legitimate and malicious content, which results in increased risks of data breaches, financial losses, and system compromise.

Traditional cybersecurity solutions, including antivirus software and online scanning tools, are often complex, expensive, or designed for technically skilled users. These systems typically require installation, subscription fees, and prior knowledge, making them inaccessible to a large portion of the population. Moreover, most existing platforms focus on a single type of threat and fail to provide a unified solution for detecting phishing, malicious URLs, and malware in one system.

Therefore, there is a need for an intelligent, accessible, and user-friendly cybersecurity platform that can detect multiple types of threats in real time and present results in a clear and understandable manner for non-technical users.

2.2 Deliverables and Development Requirements

The key deliverables of CyberSentinel AI include:

- A fully functional web-based cybersecurity application
- Phishing email detection module using NLP and machine learning
- Malicious URL analysis module with risk scoring
- File malware detection module integrated with VirusTotal API
- User-friendly dashboard with visual threat indicators
- Report generation and download functionality
- Scan history and logging system
- Integration with external threat intelligence APIs (VirusTotal, PhishTank, AbuseIPDB)

The development requirements of the system include:

- Frontend development using React.js
- Backend API development using Flask
- Machine learning implementation using scikit-learn
- Natural Language Processing using TF-IDF techniques
- Integration of external APIs for enhanced threat detection
- Standard computing environment with internet connectivity
- Compatibility with modern web browsers (Chrome, Firefox, Edge)
- Secure communication using HTTPS protocols

2.3 Current System

Existing cybersecurity solutions primarily rely on traditional antivirus software and online scanning platforms. While these systems provide basic protection, they have several limitations in terms of usability, accessibility, and scope.

Limitations of Current Systems:

- Most tools focus only on malware detection and do not effectively handle phishing or malicious URLs
- Lack of a unified platform, requiring users to use multiple tools for different threats
- Complex interfaces that are difficult for non-technical users to understand
- Dependence on paid subscriptions, making them less accessible to general users
- Requirement of installation and system resources for antivirus software
- Limited real-time detection capabilities in some tools
- Difficulty in interpreting detailed technical reports provided by platforms like VirusTotal

- Lack of user-friendly visualization and simplified results
- Dependence on user judgment due to fragmented analysis

These limitations highlight the need for a unified, AI-driven cybersecurity solution like CyberSentinel AI, which provides real-time threat detection through a simple and accessible web interface.

3. Requirement Analysis

This chapter defines the functional and non-functional requirements of the CyberSentinel AI system. It explains how users interact with the system through use case modeling and describes the expected behavior of each module. The purpose of this chapter is to provide a clear and structured specification of system requirements for design and implementation.

3.1 Use Case Diagram

The use case diagram provides a high-level representation of the interactions between system actors and CyberSentinel AI. The primary actors include the **User** and **Admin**.

The User interacts with the system to perform threat analysis tasks such as scanning emails, analyzing URLs, uploading files, viewing results, and downloading reports. The Admin manages system operations including user management, system monitoring, and log analysis.

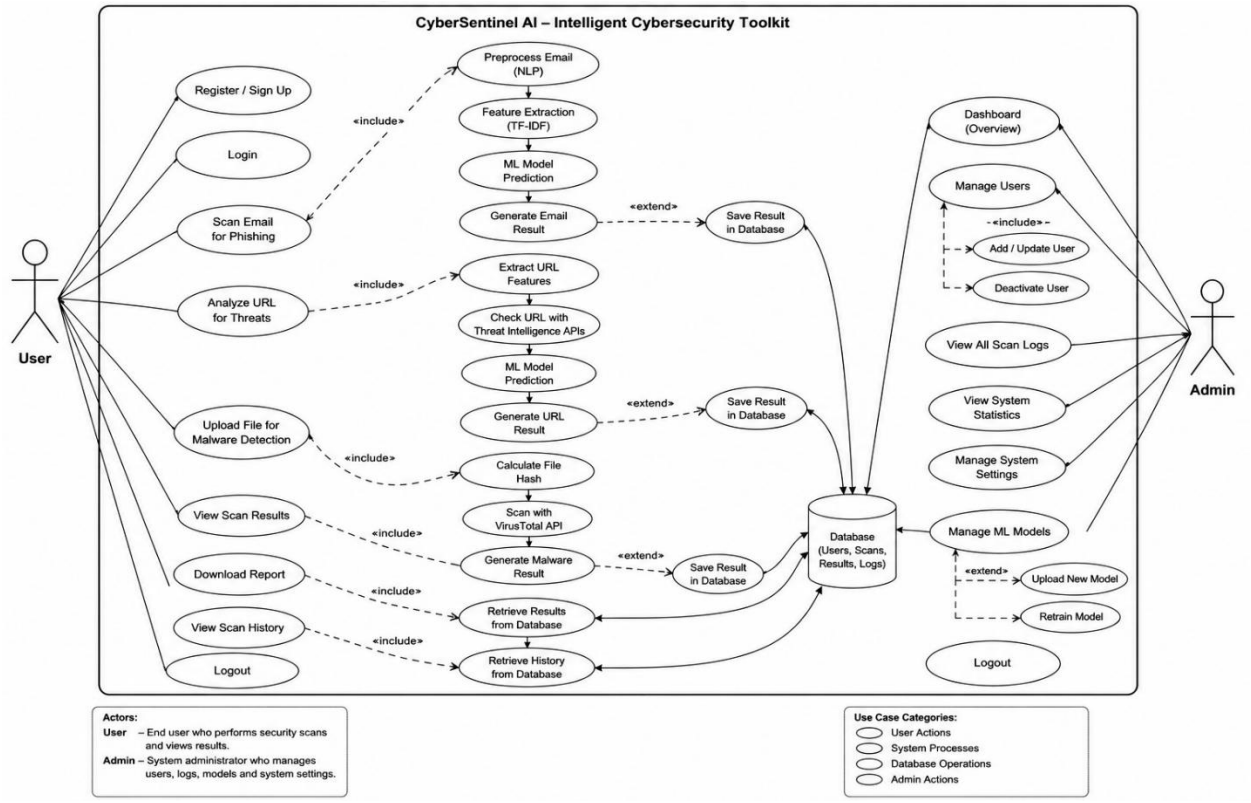

Figure 3. 1: Use Case Diagram

3.2 Detailed Use Cases

Table 3. 1: USE CASE

Field:	**Description**
Use Case ID:	UC-01
Use Case Name:	Scan Email for Phishing

Primary Actor:	User
Secondary Actors:	Backend API, NLP Module, ML Model
Description:	User submits email text for phishing detection
Trigger:	User clicks "Analyze"
Preconditions:	Internet available, system running, model loaded
Postconditions:	Prediction displayed, result stored in logs

Description – Scan Email for Phishing

Normal Flow:

1. User enters email text
2. User clicks analyze
3. System validates input
4. NLP preprocessing applied
5. ML model generates prediction
6. Result displayed with confidence score

Alternative Flow:

- Empty input → error message
- Server unavailable → failure notification

Table 3. 2: Use Case Description – Scan URL for Threats

Field	Description
Use Case ID	UC-02
Use Case Name	Analyze URL
Primary Actor	User
Secondary Actors	Backend API, ML Model, Threat APIs
Description	User submits URL for threat analysis
Trigger	User clicks "Analyze URL"
Preconditions	Internet connection, API keys active
Postconditions	Risk level displayed, result stored

Normal Flow:

1. User inputs URL
2. System validates format

3. ML model analyzes URL
4. APIs provide threat intelligence
5. Risk score generated
6. Result displayed

Table 3. 3: Use Case Description – Malware File Detection

Field	**Description**
Use Case ID	UC-03
Use Case Name	File Malware Detection
Primary Actor	User
Secondary Actors	Backend API, VirusTotal API
Description	User uploads file for malware scanning
Trigger	User uploads file
Preconditions	Valid file, API access available
Postconditions	Malware result displayed

Normal Flow:

1. User uploads file
2. System validates file
3. File hash generated
4. API request sent
5. Results retrieved
6. Output displayed

3.3 Functional Requirements

The functional requirements define the expected behavior of the system.

Table 3. 4: Functional Requirements

ID	Requirement
FR-01	System shall allow users to submit email text
FR-02	System shall classify email using NLP and ML
FR-03	System shall allow URL submission
FR-04	System shall analyze URLs using ML and APIs
FR-05	System shall allow file uploads
FR-06	System shall integrate VirusTotal API

ID	Requirement
FR-07	System shall display results with color indicators
FR-08	System shall store scan history
FR-09	System shall generate downloadable reports
FR-10	System shall support admin functionalities

3.4 Non-Functional Requirements

Table 3. 5: Non-Functional Requirements

Category	Requirement
Usability	System shall provide simple UI for non-technical users
Performance	Response time should be within a few seconds
Security	Data must be transmitted via HTTPS

Category	Requirement
Reliability	System should handle failures gracefully
Maintainability	System should support modular updates

3.5 UML Diagrams

The system design is supported by UML diagrams to visually represent structure and behavior.

The following diagrams will be included:

- Use Case Diagram
- Class Diagram
- Sequence Diagram
- Activity Diagram
- Entity Relationship Diagram
- *CyberSentinel AI – Intelligent Cybersecurity Toolkit*

4. Design and Architecture

This chapter presents the overall system design and architecture of CyberSentinel AI. It explains the structural organization of the system and illustrates how different components interact to perform cybersecurity threat detection. The chapter also includes UML diagrams and database design used for system implementation.

4.1 System Architecture

CyberSentinel AI follows a modular three-tier architecture consisting of the frontend layer, backend layer, and AI/ML processing layer. This architecture ensures scalability, maintainability, and efficient communication between system components.

The frontend layer is developed using React.js and provides a responsive user interface through which users can interact with the system. Users can submit suspicious emails, URLs, or files and view threat analysis results in real time.

The backend layer is implemented using the Flask framework. It handles API requests, validates user inputs, communicates with machine learning modules, and manages interactions with external threat intelligence APIs such as VirusTotal, PhishTank, and AbuseIPDB.

The AI/ML layer contains the phishing detection models, NLP preprocessing pipelines, and URL analysis mechanisms. TF-IDF vectorization and Logistic Regression are used for phishing email classification, while external APIs and machine learning techniques are used for URL and malware analysis.

The database layer stores user accounts, scan history, logs, and generated reports. This layer ensures persistent storage and supports system monitoring and audit functionality.

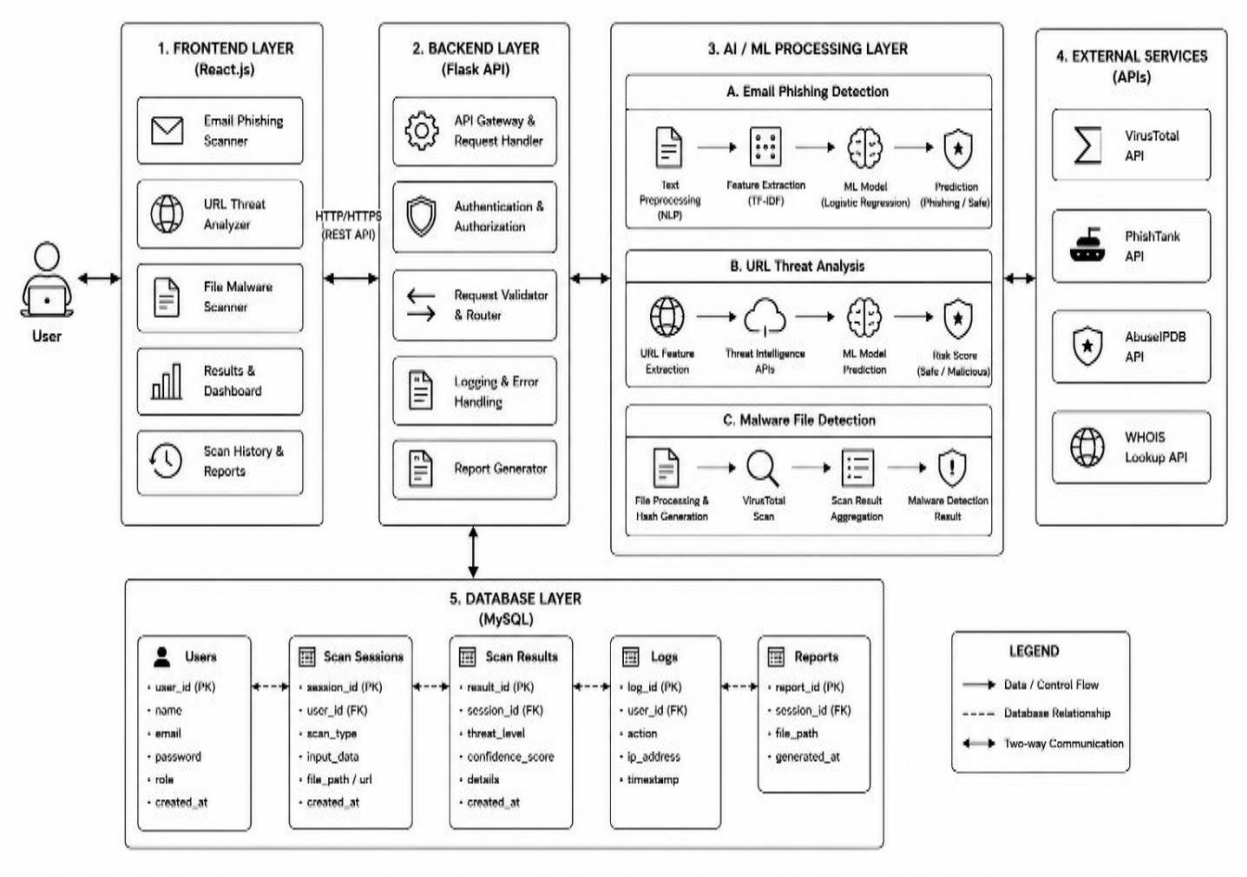

Figure 4. 1: System Architecture Diagram

4.2 Entity Relationship Diagram

The Entity Relationship (ER) diagram represents the database structure of CyberSentinel AI. It illustrates the entities used in the system along with their attributes and relationships.

The primary entities include User, Scan Session, Scan Result, Logs, and Reports. A user can perform multiple scan sessions, while each session generates a corresponding scan result. Logs are maintained for monitoring system activities and reports are generated for storing downloadable scan summaries.

The ER diagram helps define the logical database design and ensures proper organization and retrieval of system data.

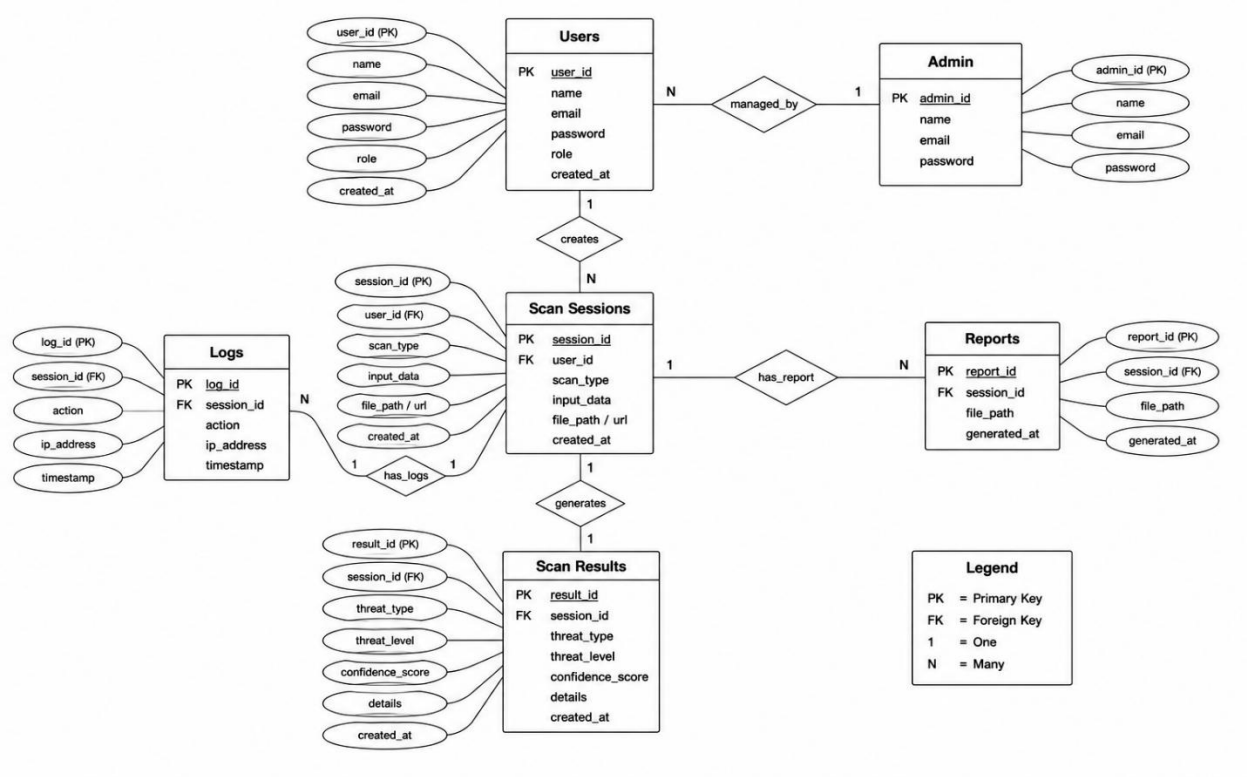

Figure 4. 2: Entity Relationship Diagram

4.3 Process Flow Diagram

The process flow diagram illustrates the workflow of the CyberSentinel AI system. It describes how the system processes user input and generates threat analysis results.

The process begins when the user submits an email, URL, or file to the system. Based on the input type, the request is forwarded to the corresponding analysis module. Email content is processed using NLP preprocessing and machine learning classification, URLs are analyzed using feature extraction and threat intelligence APIs, and uploaded files are scanned through VirusTotal API integration.

After analysis is completed, the generated results are stored in the database and displayed to the user with confidence scores, risk indicators, and recommendations. Users can also view scan history and download reports.

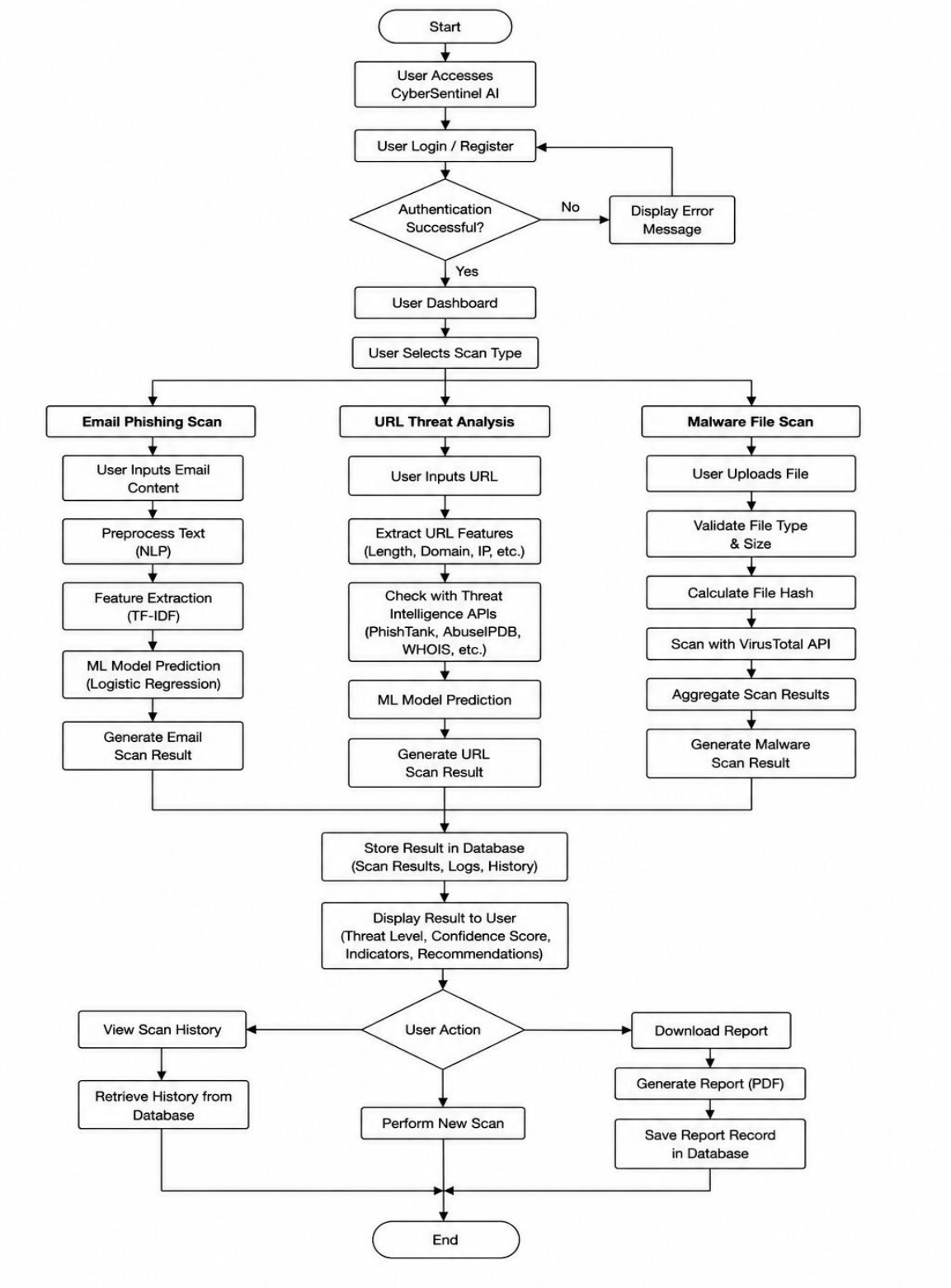

Figure 4. 3: Process Flow Diagram

4.4 Class Diagram

The class diagram represents the structural design of the system and shows the relationships between system classes. It identifies the main classes, their attributes, methods, and associations used within CyberSentinel AI.

The primary classes include User, Admin, ScanSession, ScanResult, and ThreatDetection modules. The User class manages user operations and authentication, while the Admin class handles system monitoring and management functionalities.

The ScanSession class stores information related to user scans, whereas the ScanResult class maintains prediction outputs, confidence scores, and threat levels. Additional classes are responsible for machine learning processing and API communication.

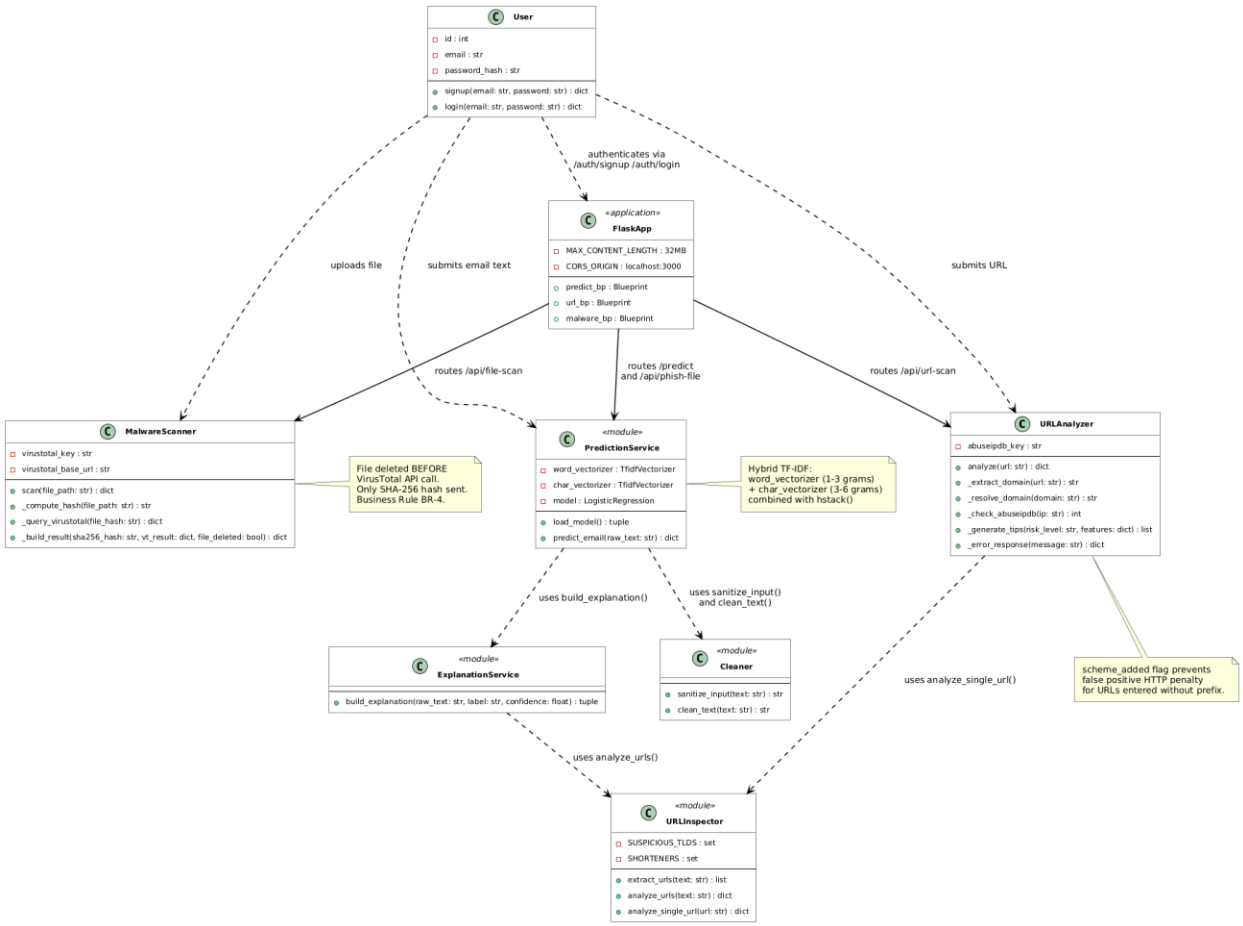

Figure 4. 4: Class Diagram

4.5 Sequence Diagram

The sequence diagram illustrates the interaction flow between the user, frontend interface, backend server, machine learning modules, and database during the threat analysis process.

When a user submits data for scanning, the frontend sends a request to the backend API. The backend validates the request and forwards it to the appropriate processing module. After analysis is completed, the result is returned to the backend, stored in the database, and displayed to the user through the frontend interface.

The sequence diagram highlights the communication and synchronization between different system components involved in the threat detection workflow.

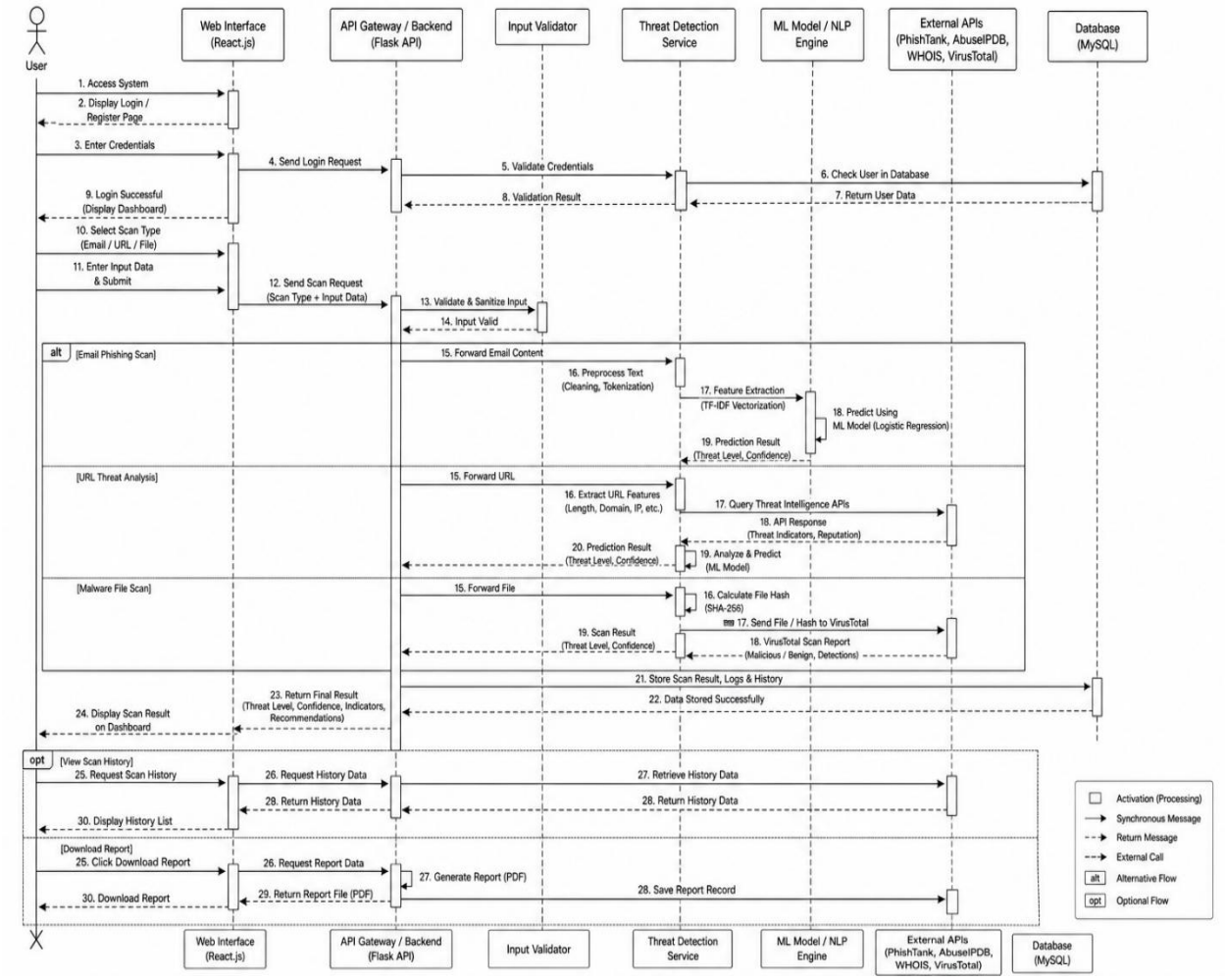

Figure 4. 5: Sequence Diagram

5. Implementation

This chapter explains the implementation details of CyberSentinel AI – Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. It describes the main algorithms used in the project, the external APIs integrated into the system, and the user interface screens developed for different users. The implementation of CyberSentinel AI is based on a modular approach in which each major functionality is developed as a separate component and then integrated into a unified web-based cybersecurity platform.

The final implementation includes phishing email detection, embedded link scanning, hidden .eml link analysis, malicious URL analysis, malware file scanning, VirusTotal live upload for unknown files, scan history, report generation, user authentication, admin panel, email notifications, Gmail/Chrome extension, and live deployment. The system uses a React.js frontend, Flask backend, machine learning models, SQLite database, and external threat intelligence APIs to provide real-time cybersecurity analysis.

The implementation follows the system architecture discussed in Chapter 4. The frontend layer handles user interaction, the backend layer processes requests and controls system logic, the AI/ML layer performs threat classification, the external API layer provides live threat intelligence, and the database layer stores user records, scan history, logs, and reports.

5.1 Algorithm

This section explains the major algorithms and workflows implemented in CyberSentinel AI. The purpose of this section is to describe how the main system modules operate without including source code. Each algorithm is explained using step-by-step natural language so that the working of the system can be clearly understood.

5.1.1 Phishing Email Detection Algorithm

The phishing email detection module is one of the core modules of CyberSentinel AI. It analyzes email text using Natural Language Processing and Machine Learning techniques. The module classifies email content as Safe, Suspicious, or Phishing and provides a confidence score with explanation. The phishing detection system uses a hybrid TF-IDF approach with Logistic Regression. Word-level TF-IDF captures complete words and phrases commonly found in phishing messages, while character-level TF-IDF helps detect obfuscated words and suspicious character patterns.

Algorithm Steps:

1. User enters email text or uploads an email file through the frontend.
2. The frontend sends the email content to the Flask backend.
3. The backend validates the input to ensure that the email text is not empty.
4. The input is sanitized to remove harmful HTML tags, scripts, and unnecessary characters.

5. Text preprocessing is applied, including lowercasing, punctuation removal, URL tokenization, and spacing normalization.

6. Word-level TF-IDF vectorization is applied to capture important phishing words and phrases.

7. Character-level TF-IDF vectorization is applied to capture obfuscated patterns.

8. Both feature matrices are combined into a single feature vector.

9. The trained Logistic Regression model predicts the class of the email.

10. The system calculates the confidence score and prepares threat indicators.

11. The final result is returned to the frontend and stored in scan history.

12. If the result is suspicious or phishing, the notification module may generate an alert.

Output of the module includes:

- Email classification result
- Confidence score
- Threat indicators
- Safety recommendations
- Timestamp
- Scan history record

5.1.2 Embedded Link Scanning Algorithm

The embedded link scanning module extracts visible URLs from email text and analyzes each URL individually using the URL analyzer. This improves phishing detection because many phishing emails contain malicious links even when the surrounding message appears normal.

Algorithm Steps:

1. User submits email text for phishing analysis.

2. The backend scans the email text using a URL extraction pattern.

3. All visible URLs found in the email are collected in a list.

4. Duplicate URLs are removed to avoid repeated scanning.

5. Each extracted URL is passed to the URL analyzer module.

6. The URL analyzer checks structural URL features and reputation information.

7. Each link receives an individual verdict as Safe, Suspicious, or Malicious.

8. The embedded link results are combined with the main phishing email result.

9. The frontend displays a separate embedded links section.

10. The scan record is stored in the database.

5.1.3 Hidden Link Detection in .eml Files

The final version implements hidden link detection for .eml email files. This is an advanced feature because hidden links cannot always be detected from copied plain text. In HTML emails, the visible text may show a trusted phrase while the actual hyperlink points to a malicious domain.

Algorithm Steps:

1. User uploads a .eml email file.

2. The backend reads the email file using an email parsing module.

3. The system separates plain text and HTML content from the email body.

4. HTML content is parsed to identify anchor tags.

5. For each anchor tag, the system extracts visible link text and hidden href destination.

6. The destination URL is passed to the URL analyzer.

7. The system checks whether the visible text and destination domain are mismatched.

8. If the visible text appears safe but the actual link points to a suspicious or unrelated domain, the system flags it as a hidden-link risk.

9. The result is added to the email scan output.

10. The frontend displays hidden link warnings separately and stores the scan result in history.

5.1.4 URL Threat Analysis Algorithm

The URL threat analyzer checks suspicious URLs using structural features and threat intelligence. It classifies URLs as Safe, Suspicious, or Malicious. The analyzer evaluates URL length, IP-based domains, @ symbols, HTTP usage, suspicious keywords, hyphens, shorteners, excessive subdomains, suspicious top-level domains, and long paths.

Algorithm Steps:

1. User submits a URL through the URL scan page or through an embedded email link.

2. The backend validates the URL format and normalizes it if necessary.

3. The system extracts structural URL features such as length, domain format, suspicious keywords, subdomain count, scheme, and path length.

4. Each suspicious feature contributes to a risk score.

5. The system resolves the domain to an IP address where possible.

6. The resolved IP is checked using AbuseIPDB.

7. If PhishTank lookup is available, the URL reputation is checked against known phishing records.

8. The local risk score and threat intelligence score are combined.

9. The system assigns a final verdict as Safe, Suspicious, or Malicious.

10. The result is displayed with risk indicators, explanation, and recommendation.

11. The URL scan result is saved in the database.

Output of the module includes:

- Final URL verdict
- Risk score
- Matched suspicious features
- Threat intelligence result
- Recommendation for the user

5.1.5 Malware File Scanning Algorithm

The malware file scanning module allows users to upload suspicious files for scanning. The system validates the file, computes its SHA-256 hash, and checks it through VirusTotal. File scanning is designed with privacy in mind because files are handled temporarily and are not permanently stored.

Algorithm Steps:

1. User uploads a file through the file scan page.

2. The frontend sends the file to the Flask backend using multipart form data.

3. The backend validates file type, file size, and file name safety.

4. The file is temporarily saved in a secure temporary location.

5. The system calculates the SHA-256 hash of the file.

6. The file is checked against VirusTotal using hash lookup.

7. If VirusTotal returns a known result, the system parses malicious, suspicious, harmless, undetected, and flagged-engine counts.

8. The system generates a verdict as Clean, Suspicious, or Malicious.

9. The file is deleted after processing according to the privacy policy.

10. The result is returned to the frontend and stored as scan metadata.

11. If the file is suspicious or malicious, an email notification may be sent.

5.1.6 VirusTotal Live Upload for Unknown Files

A limitation of hash-based scanning is that a new file may not exist in the VirusTotal database. The final system improves this by uploading unknown files to VirusTotal for live analysis and polling until the result becomes available.

Algorithm Steps:

1. File hash lookup is performed through VirusTotal.
2. If the file is found, the system uses the returned report.
3. If the file is not found, the system marks it as unknown temporarily.
4. The backend uploads the file to VirusTotal's file analysis endpoint.
5. VirusTotal returns an analysis ID.
6. The backend waits for a short interval and checks the analysis status using the analysis ID.
7. If the analysis is completed, the result is fetched and parsed.
8. If the maximum polling limit is reached, the system returns a pending or delayed result.
9. The final verdict is displayed to the user and saved in scan history.
10. The user may receive an email notification if the verdict is suspicious or malicious.

5.1.7 Scan History and Database Storage Algorithm

The scan history module stores previous scans so that users can review earlier results. This improves system usability and provides evidence for later reference.

Algorithm Steps:

1. User performs an email, URL, or file scan.
2. The backend receives the result from the related detection module.
3. The system creates a scan session record.
4. The scan result is saved with user ID, scan type, threat type, threat level, confidence score, timestamp, and result details.
5. If a report is generated, the report path is linked with the scan session.
6. The history page requests previous scans for the logged-in user.
7. The backend retrieves records from the database.
8. The frontend displays the scan history in tabular or card format.
9. Users can view details or download available reports.

5.1.8 Report Generation Algorithm

The report generation module allows users to download a summary of scan results. Reports are useful for record keeping, awareness, and sharing suspicious content analysis with other people.

Algorithm Steps:

1. User completes a scan.
2. User clicks the Download Report button.
3. The frontend collects the current scan result.
4. The report generator prepares report content including scan type, verdict, confidence score, threat indicators, recommendations, date, and time.
5. The report is generated in PDF format.
6. The file is downloaded by the user.
7. A report record is saved in the database.

5.1.9 Admin Panel Workflow

The admin panel allows administrative monitoring and management of the system. The admin can view users, scan records, logs, system statistics, and threat activity.

Algorithm Steps:

1. Admin logs into the system using admin credentials.
2. The backend verifies the admin role.
3. After successful login, the admin dashboard is displayed.
4. The admin can view total users, total scans, email scans, URL scans, file scans, threat statistics, and recent logs.
5. The admin can manage users where required.
6. The admin can review scan history and audit logs.
7. System statistics are updated from database records.
8. Admin actions are logged for traceability.

5.1.10 Email Notification Workflow

The email notification module sends alerts or scan summaries to users when important events occur. It is especially useful when a scan result is suspicious or malicious.

Algorithm Steps:

1. A scan is completed.
2. The system checks the final threat level.

3. If the result is suspicious or malicious, the notification module is triggered.

4. The system prepares an email message containing scan type, threat verdict, confidence score, short explanation, and safety recommendation.

5. The email is sent to the registered user using SMTP or configured email service.

6. Notification status is recorded in logs.

7. If email sending fails, the system records the error and continues without interrupting the scan result display.

5.1.11 Gmail/Chrome Extension Workflow

The Gmail/Chrome extension allows users to scan email content directly from Gmail. This makes the system more practical because users do not always need to copy and paste email text manually.

Algorithm Steps:

1. User opens Gmail in the Chrome browser.

2. User opens a suspicious email.

3. User clicks the CyberSentinel AI extension icon.

4. The extension content script reads the visible email text and available links.

5. The extension sends the extracted email data to the live CyberSentinel backend.

6. The backend runs phishing detection and embedded link analysis.

7. The backend returns the verdict and confidence score.

8. The extension popup displays the result to the user.

9. If the email is risky, the user receives a warning.

10. The scan may be saved in scan history if the user is authenticated.

5.1.12 Live Deployment Workflow

The final version of CyberSentinel AI is deployed so that it can be accessed beyond the local development environment.

Algorithm Steps:

1. Flask backend is deployed on a cloud hosting platform.

2. Environment variables are configured securely on the hosting platform.

3. API keys are stored on the server side only.

4. React frontend is deployed on a static hosting platform.

5. Frontend API base URL is changed from localhost to the live backend URL.

6. CORS settings are updated to allow the deployed frontend domain.
7. Database and logs are configured for the deployed environment.
8. All modules are tested from a different device and network.
9. The Gmail extension is connected to the live backend.

5.2 External APIs

CyberSentinel AI integrates external APIs to improve real-time threat intelligence and detection accuracy. These APIs allow the system to check URLs, IP addresses, and files against large external cybersecurity databases. API keys are stored on the backend side and are not exposed to the frontend or browser extension.

Table 5. 1: Details of APIs Used in the Project

Name of API	Description of API	Purpose of Usage	Function/Class Name in Which It Is Used
VirusTotal API v3	VirusTotal is a threat intelligence platform that analyzes files and URLs using multiple antivirus engines and security vendors.	Used for malware file scanning, SHA-256 hash lookup, live upload of unknown files, and retrieval of file analysis results.	MalwareScanner, malware_service.py, file_scan_route
AbuseIPDB API	AbuseIPDB provides reputation information about IP addresses reported for malicious or abusive activity.	Used to check the reputation of IP addresses resolved from submitted URLs.	URLAnalyzer, url_service.py, abuseipdb_lookup
PhishTank API	PhishTank provides a community-based database of reported phishing URLs.	Used to verify whether a submitted URL is already reported as a phishing website.	URLAnalyzer, phishtank_lookup
SMTP / Email Service	SMTP or configured email service is used to send system generated email messages to users.	Used for sending threat alerts, suspicious scan notifications, and scan summary messages.	NotificationService, email_service.py
CyberSentinel Backend API	Internal REST API developed using Flask for communication between the frontend, extension, and backend modules.	Used by React frontend and Gmail extension to submit scans and receive results.	app.py, predict_routes.py, url_routes.py, malware_routes.py
Gmail / Chrome Extension API	Browser extension interface used to interact with Gmail page content and browser popup components.	Used to extract email text and links from Gmail and send them to the CyberSentinel live backend.	manifest.json,content.js, popup.js

The external API integration makes CyberSentinel AI more reliable because the system does not depend only on local rules or machine learning predictions. Instead, it combines local AI-based analysis with live global threat intelligence.

5.3 User Interface

The user interface of CyberSentinel AI is designed to be simple, clear, and understandable for non-technical users. Since the target users include students, freelancers, small business owners, and general internet users, the interface avoids unnecessary technical complexity and presents results in plain language.

The system uses a dashboard-based interface developed in React.js. Users can navigate to different modules from the sidebar or navigation menu. Each module follows a simple workflow: submit input, click scan, view result, and download report if required.

5.3.1 Login and Registration Interface

The login and registration interface allows users to create an account and access the system securely. Registered users can scan content, view scan history, download reports, and receive email notifications.

Main features include:

- User registration
- User login
- Role-based access
- Session handling
- Logout functionality

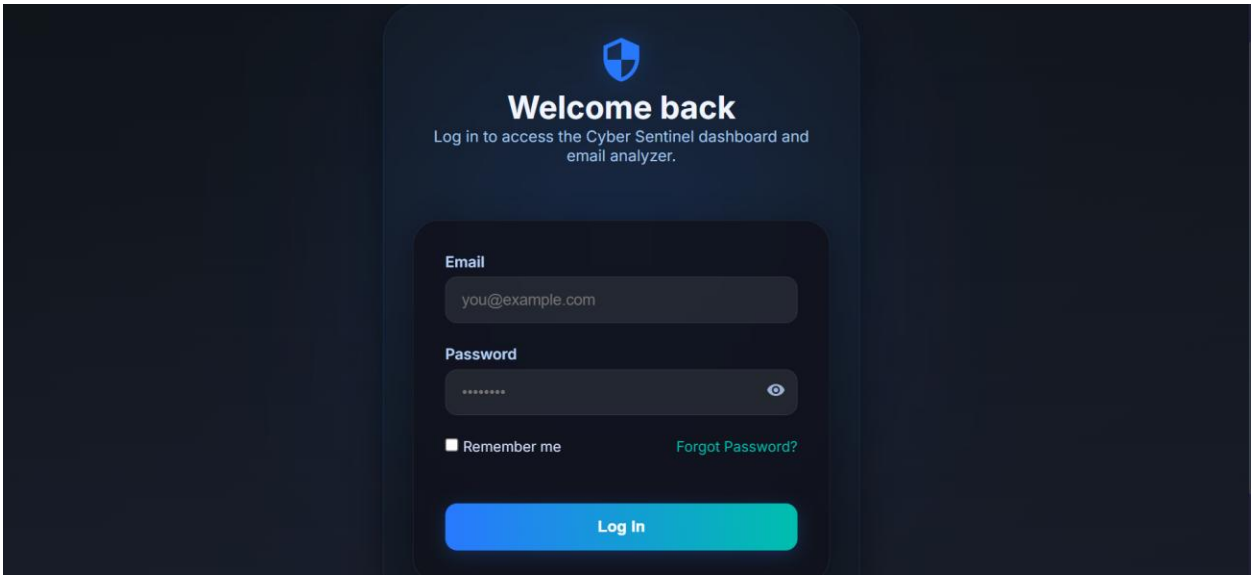

Figure 5. 1: Login and Registration Interface

5.3.2 User Dashboard

The dashboard provides an overview of the system and gives users access to all major scanning modules. It may also show scan statistics such as total scans, recent scans, and detected threats.

This section shows:

- Email phishing scan
- URL threat analysis
- Malware file scan
- Scan history
- Download reports
- User profile
- Logout

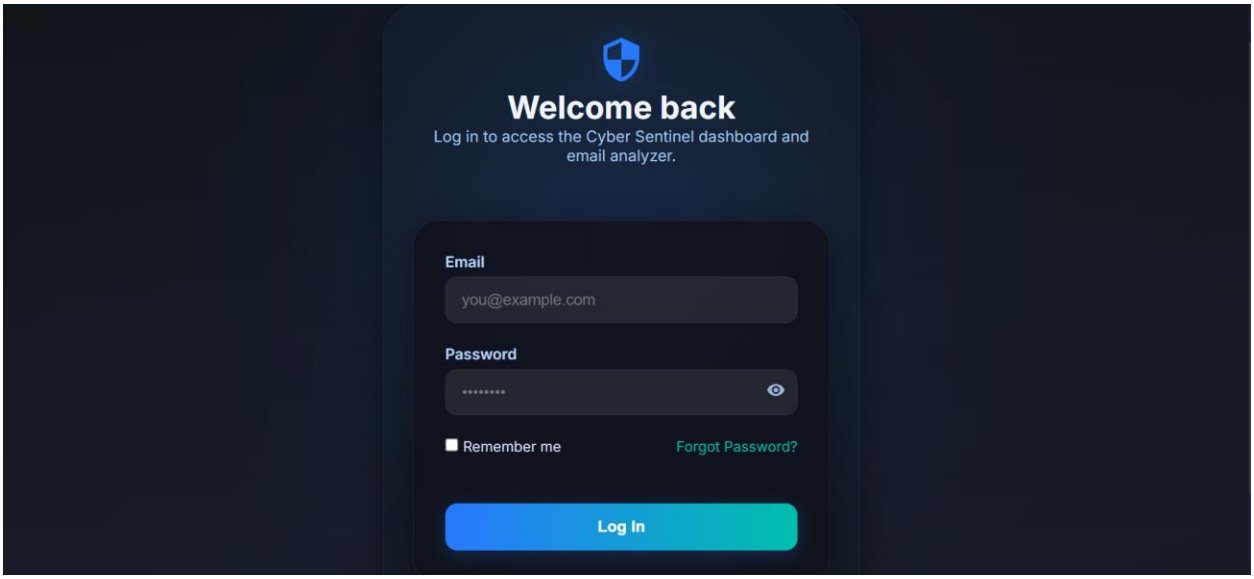

Figure 5. 2: User Dashboard

5.3.3 Email Phishing Scan Interface

The email scan page allows users to paste suspicious email text or upload email files. After scanning, the system displays whether the email is safe, suspicious, or phishing.

Main features include:

- Threat verdict
- Confidence score
- Explanation
- Embedded link analysis
- Hidden .eml link warning
- Recommended action

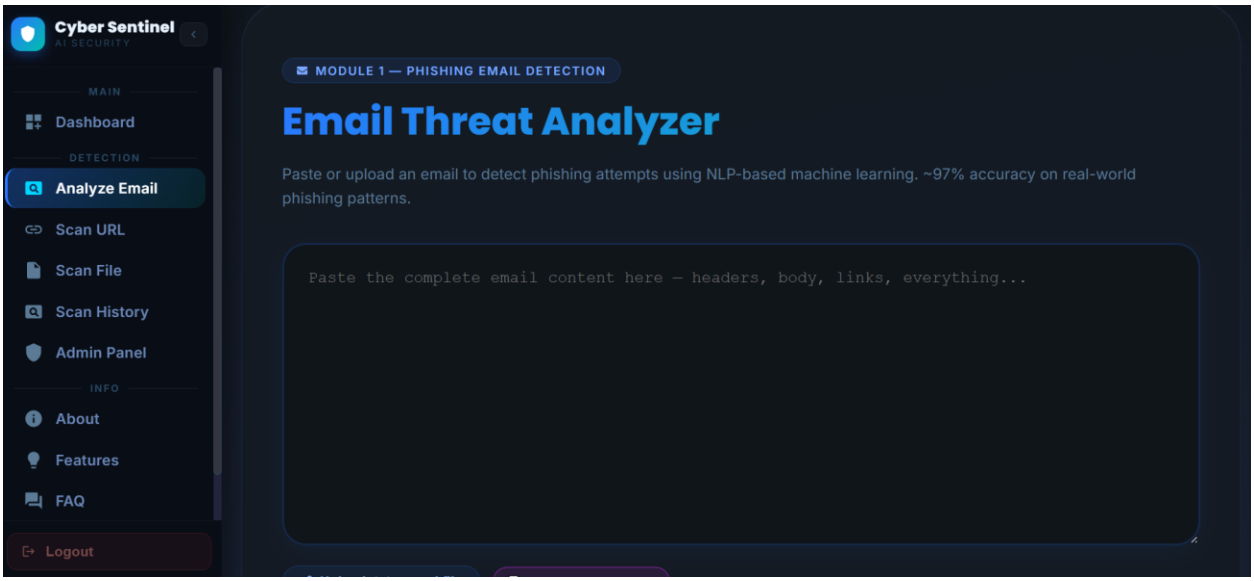

Figure 5. 3:Email Phishing Scan Interface

5.3.4 Embedded and Hidden Link Result Interface

The embedded link result section displays all extracted URLs found inside the email. Each URL is analyzed separately and shown with its own risk level. For .eml files, hidden HTML links are also displayed if the visible text and actual destination are mismatched.

This section shows:

- Extracted visible URL
- Hidden destination URL
- URL verdict
- Mismatch warning

• Risk score

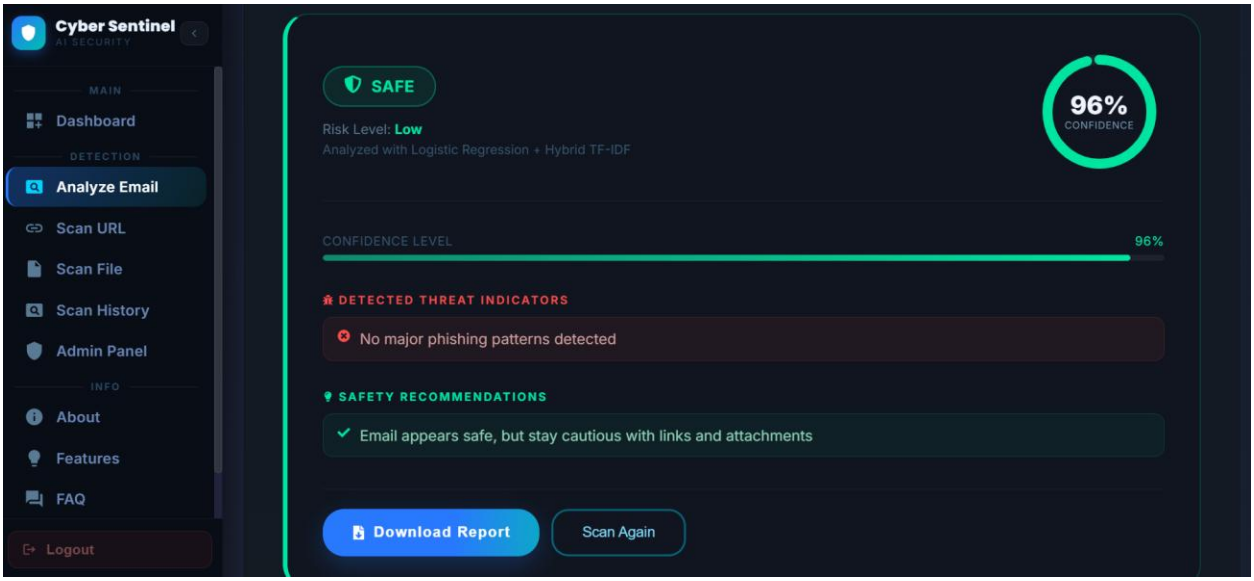

Figure 5. 4: Embedded and Hidden Link Detection Result

5.3.5 URL Threat Analysis Interface

The URL scan page allows users to enter suspicious links. The system checks URL structure and threat intelligence results.

Main features include:

• URL verdict

• Risk score

• Suspicious features

• AbuseIPDB result

• PhishTank result

• Final recommendation

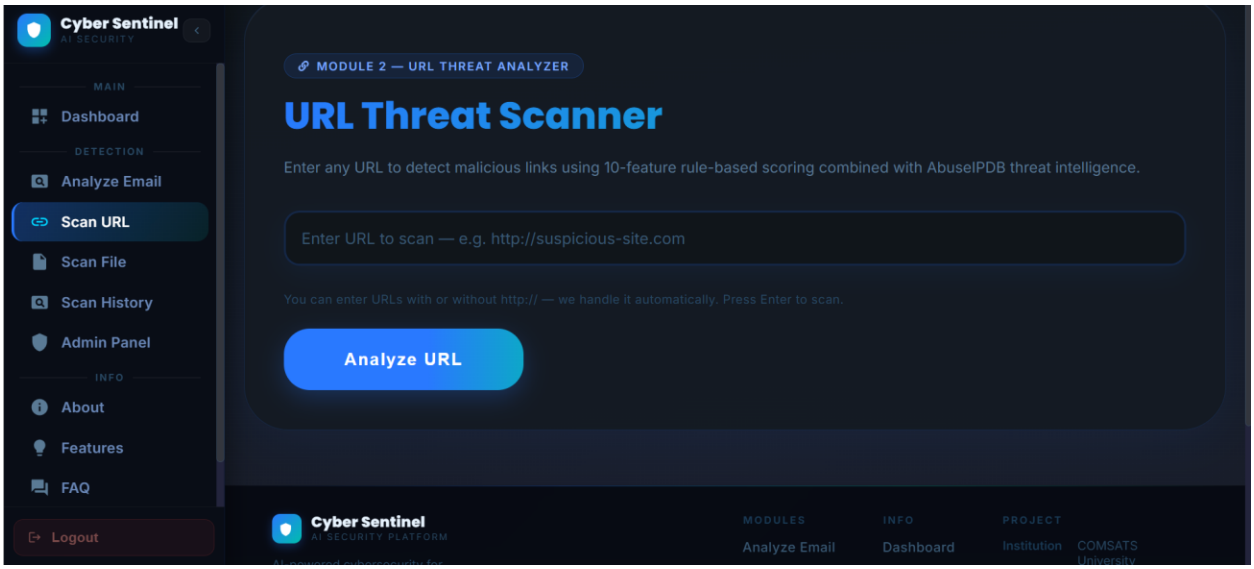

Figure 5. 5:URL Threat Analysis Interface

5.3.6 Malware File Scan Interface

The file scan page allows users to upload files for malware scanning. The system validates the file and scans it using VirusTotal.

Main features include:

- File name
- File hash
- VirusTotal result
- Malicious engine count
- Suspicious engine count
- Final verdict
- File privacy status

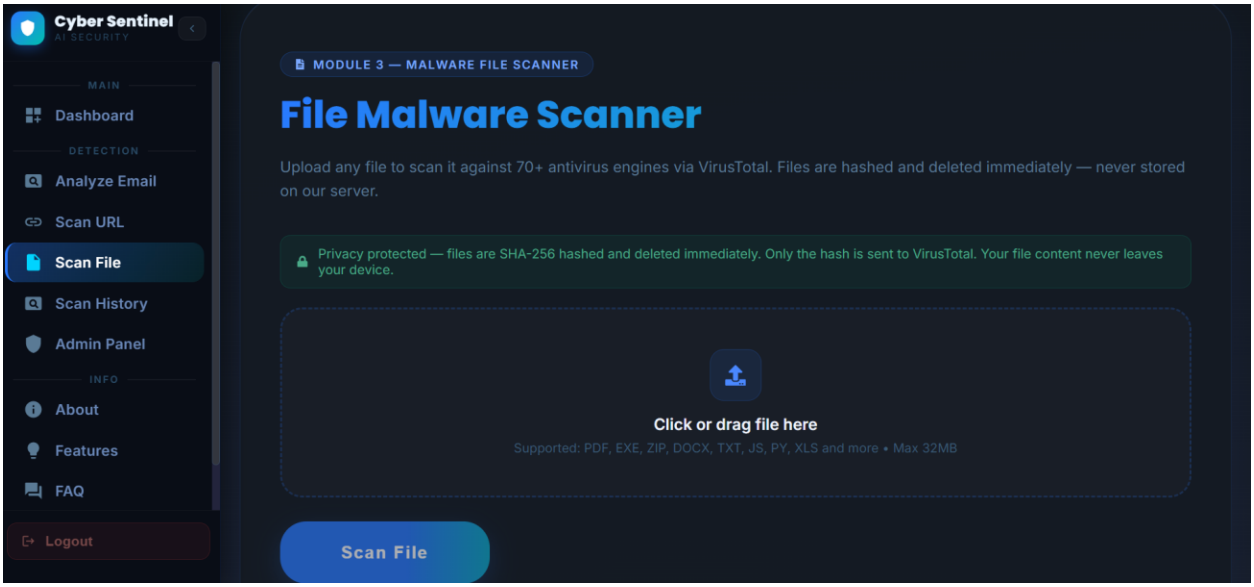

Figure 5. 6: Malware File Scan Interface

5.3.7 Scan History Interface

The scan history page displays previous scans performed by the user. This allows users to review older results and download reports if available.

Main features include:

- Scan type
- Date and time
- Threat verdict
- Confidence score
- Report option
- Detailed view

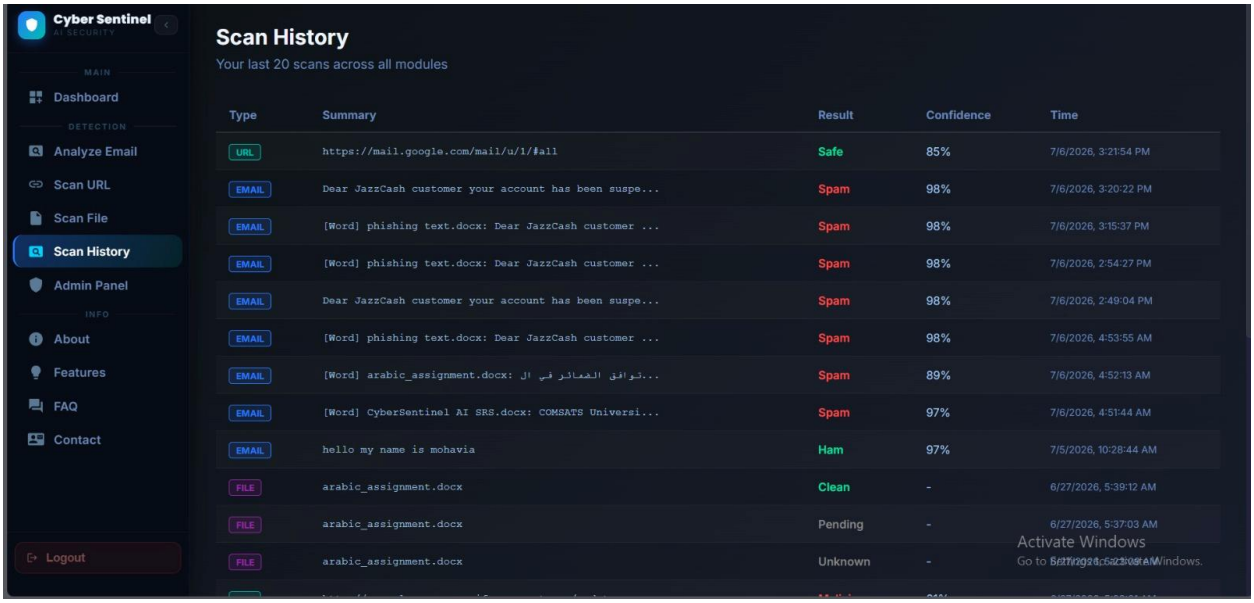

Figure 5. 7: Scan History Interface

5.3.8 Report Interface

The report interface allows users to generate and download scan reports. Reports summarize the scan result in a structured format.

Main features include:

- Project name
- Scan type
- Verdict
- Confidence score
- Threat details
- Safety tips
- Timestamp

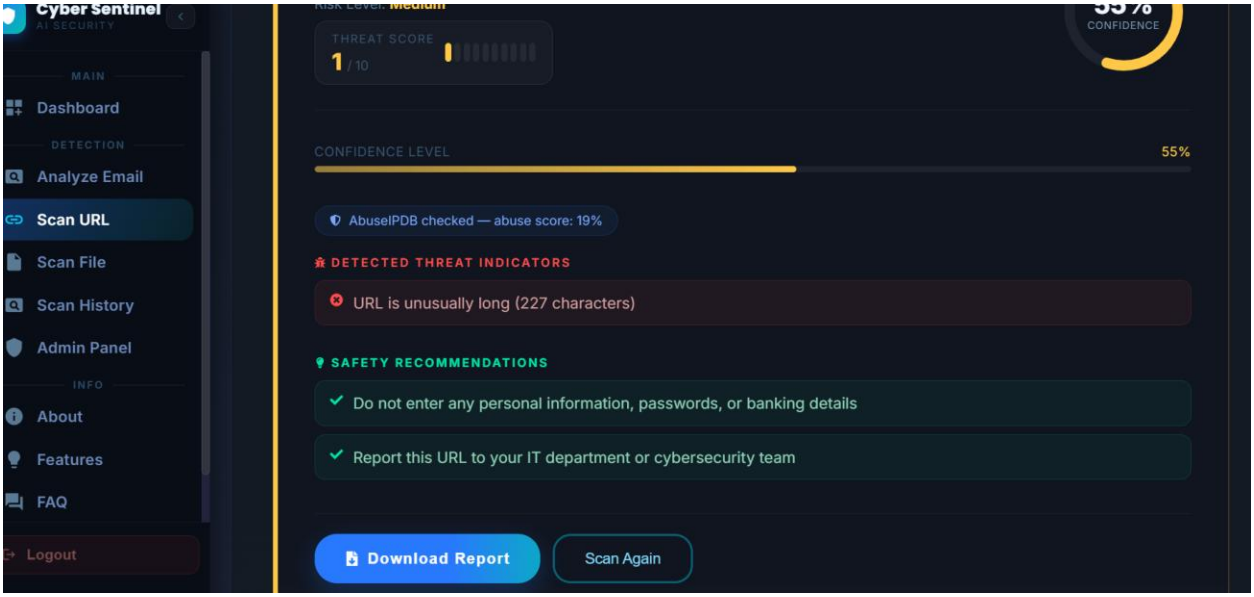

Figure 5. 8: PDF Report Interface

5.3.9 Admin Panel Interface

The admin panel is designed for system monitoring and management. Admin users can view scan statistics, user records, threat logs, and system activity.

Main features include:

- View total users
- View total scans
- View detected threats
- Manage users
- View logs
- View scan records
- Monitor system activity

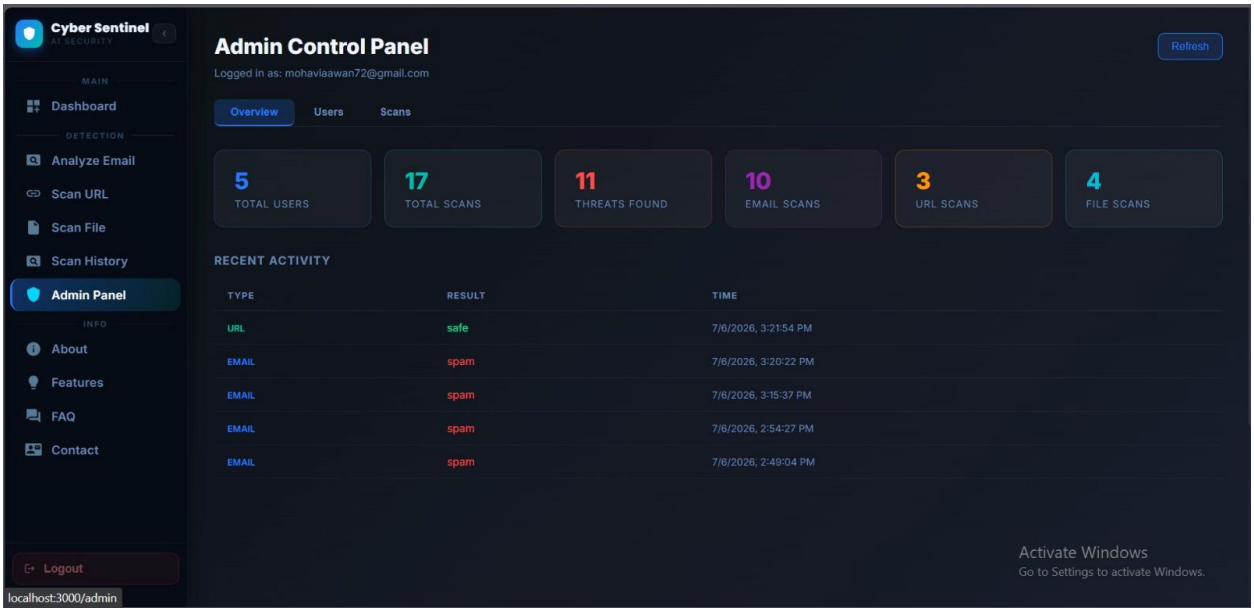

Figure 5. 9: Admin Panel Interface

5.3.10 Email Notification Output

The email notification system sends alerts to users when suspicious or malicious activity is detected. The notification contains the scan result and a short safety recommendation.

This section shows:

- Scan type
- Threat level
- Confidence score
- Short explanation
- Recommended user action

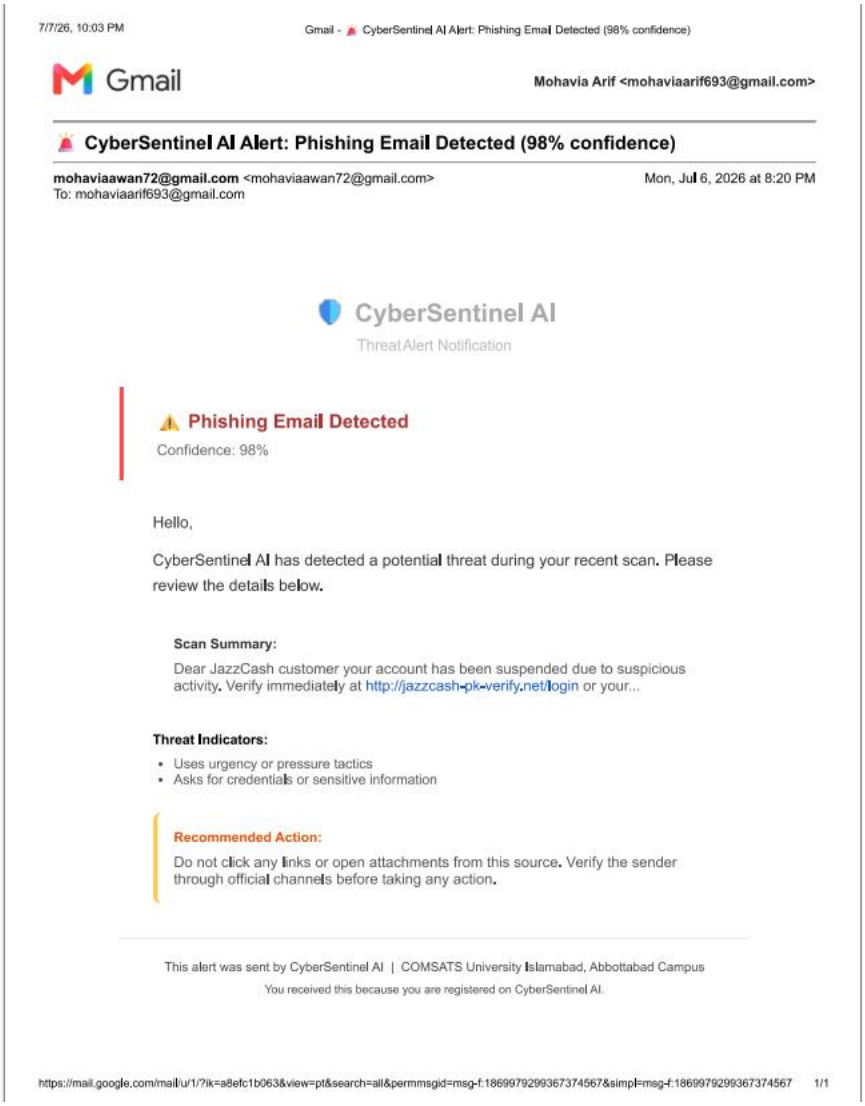

Figure 5. 10: Email Notification Output

5.3.11 Gmail/Chrome Extension Interface

The Gmail extension interface allows the user to scan an opened Gmail message directly from the browser. The extension popup displays the verdict without requiring the user to manually copy the email into the web dashboard.

Main features include:

- Scan current email button
- Result badge
- Confidence score
- Risk explanation

• Link warning

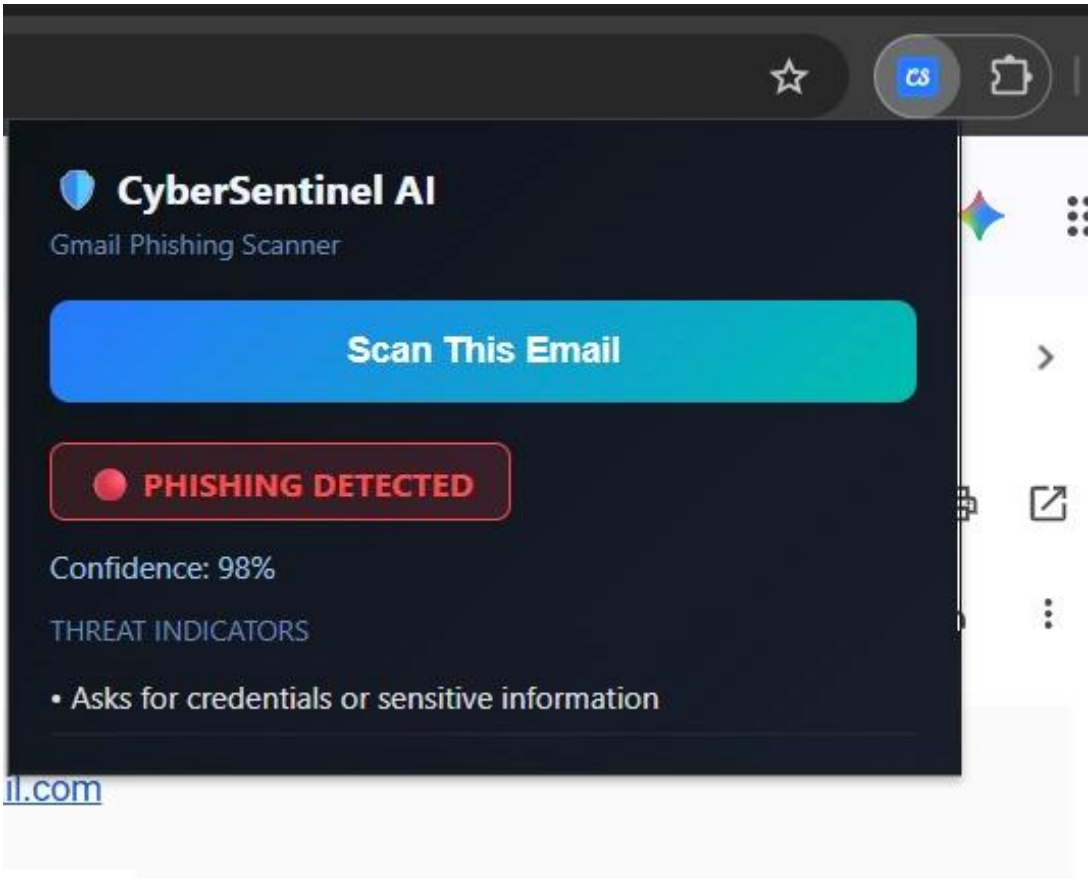

Figure 5. 11: Gmail Extension Interface

5.4 Summary

This chapter explained the implementation of CyberSentinel AI through its major algorithms, external APIs, and user interface modules. The implementation demonstrates how multiple cybersecurity features are integrated into one complete platform. The system combines machine learning, natural language processing, URL feature analysis, malware scanning, live threat intelligence, database storage, reporting, admin monitoring, email notifications, Gmail extension support, and live deployment.

The next chapter discusses the testing and evaluation of the implemented system, including manual testing, unit testing, functional testing, integration testing, and automated testing.

6. Testing and Evaluation

This chapter presents the testing and evaluation process performed on CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. Testing was conducted to verify that all implemented modules work according to the defined requirements and that the system performs reliably in real usage conditions. The testing process included manual testing, system testing, unit testing, functional testing, integration testing, automated API testing, deployment testing, and extension testing.

The final version of CyberSentinel AI includes several modules such as phishing email detection, embedded link scanning, hidden .eml link detection, malicious URL analysis, malware file scanning, VirusTotal live upload for unknown files, scan history, report generation, admin panel, email notifications, Gmail/Chrome extension, and live deployment. Each module was tested individually and then evaluated as part of the complete integrated system.

Testing was performed to ensure that the system correctly accepts user input, validates data, processes requests through the backend, performs AI/ML or API-based analysis, stores results in the database, generates reports, sends notifications, and displays results clearly to the user. The main goal of testing was to confirm that CyberSentinel AI is accurate, usable, secure, and reliable for non-technical users.

6.1 Manual Testing

Manual testing was performed by interacting with the system through the user interface and verifying whether each feature behaved as expected. The system was tested from the perspective of both normal users and administrators. Manual testing helped identify interface issues, incorrect messages, missing validations, and integration problems that may not be detected through automated tests.

The following modules were manually tested:

- User registration and login
- Email phishing scan
- Embedded link scanning
- Hidden .eml link detection
- URL threat analysis
- Malware file scanning
- VirusTotal unknown file upload
- Scan history
- PDF report generation

- Email notifications
- Admin dashboard
- Gmail/Chrome extension
- Live deployment access

6.1.1 System Testing

System testing was performed after all modules were integrated into one complete platform. The purpose of system testing was to verify that CyberSentinel AI works as a complete cybersecurity toolkit rather than separate individual modules. The testing checked complete user journeys from login to scan submission, result display, report generation, scan history storage, and notification delivery.

Table 6. 1: System Testing

No.	Test Case / Test Script	Input / Attribute	Expected Result	Result
1	Open the deployed CyberSentinel AI web application	Live frontend URL	Homepage or login page loads successfully	Pass
2	Register a new user account	Name, email, password	User account is created successfully	Pass
3	Login with valid user credentials	Registered email and password	User dashboard is displayed	Pass
4	Login with invalid credentials	Wrong email or password	Error message is displayed	Pass
5	Submit phishing email text for scanning	JazzCash-style phishing message	System classifies email as suspicious or phishing	Pass
6	Submit safe email text for scanning	Normal meeting email	System classifies email as safe	Pass
7	Submit email containing visible URL	Email with suspicious URL	Email result and embedded URL result are displayed	Pass
8	Upload .eml file containing hidden link	HTML email with mismatched link text and href	Hidden link warning is displayed	Pass

9	Submit suspicious URL	URL containing suspicious keywords and long path	URL is classified as suspicious or malicious	Pass
10	Submit safe URL	Trusted website URL	URL is classified as safe	Pass
11	Upload known malware test file/hash sample	Supported file type	VirusTotal result is displayed	Pass
12	Upload unknown clean Word file	New .docx file	File is uploaded to VirusTotal for live analysis and result is returned	Pass
13	Generate scan report	Completed scan result	PDF report is generated and downloaded	Pass
14	View scan history	Logged-in user with previous scans	Previous scans are displayed	Pass
15	Admin login	Admin credentials	Admin dashboard is displayed	Pass
16	View admin statistics	Admin dashboard	Total scans, users, and threats are displayed	Pass
17	Send email notification after risky scan	Suspicious or malicious scan result	User receives notification email	Pass
18	Scan Gmail message using extension	Open Gmail email and click extension	Extension displays scan verdict	Pass
19	Test system from another device/network	Live deployed link	System opens and scans successfully	Pass
20	Logout from system	Logout button	User session ends successfully	Pass

System testing confirmed that the complete platform works according to the expected workflow and that all major modules are connected properly.

6.1.2 Unit Testing

Unit testing was performed on individual modules to verify that each component works correctly before integration. Each unit was tested separately using controlled input and expected output. Unit testing helped ensure that errors inside one module did not affect the rest of the system.

Unit Testing 1: Phishing Email Detection

Testing Objective: To ensure that the phishing detection model correctly classifies email text.

Table 6. 2: Phishing Email Detection Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Test phishing email classification	Your JazzCash account is suspended. Verify now.	Email classified as phishing or suspicious	Pass
2	Test safe email classification	Project meeting is scheduled for Monday.	Email classified as safe	Pass
3	Test empty email input	Empty text field	Error message displayed	Pass
4	Test long email input	Large email text	System processes or rejects safely according to limit	Pass
5	Test obfuscated phishing words	verif1cation required for acc0unt	Suspicious pattern detected	Pass

Unit Testing 2: Embedded Link Scanning

Testing Objective: To ensure that visible URLs inside email text are extracted and analyzed separately.

Table 6. 3: Embedded Link Scanning Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Extract one URL from email	Email contains one suspicious link	URL is extracted successfully	Pass
2	Extract multiple URLs from email	Email contains more than one URL	All URLs are extracted	Pass
3	Remove duplicate URLs	Same URL repeated twice	Duplicate URL is ignored	Pass
4	Scan extracted suspicious URL	Extracted phishing-style URL	URL verdict shown separately	Pass
5	Submit email without URLs	Email contains no URL	System displays email result without embedded link section	Pass

Unit Testing 3: Hidden .eml Link Detection

Testing Objective: To ensure that hidden links inside .eml files are parsed and checked.

Table 6. 4: Hidden EML Link Detection Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Upload valid .eml file	Email file with HTML content	Email is parsed successfully	Pass
2	Detect anchor tag link	HTML email with href link	Actual destination URL is extracted	Pass
3	Compare visible text with hidden URL	Visible text says JazzCash but href points to unknown domain	Mismatch warning displayed	Pass
4	Scan hidden URL	Extracted hidden link	URL analyzer returns risk result	Pass
5	Upload .eml without links	Plain email file	No hidden link warning displayed	Pass

Unit Testing 4: URL Threat Analyzer

Testing Objective: To ensure that URL analysis correctly checks structural features and API reputation.

Table 6. 5: URL Analyzer Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Test safe HTTPS URL	https://www.comsats.edu.pk	URL classified as safe	Pass
2	Test HTTP URL	http://example.com/login	HTTP risk feature detected	Pass
3	Test IP-based URL	http://192.168.1.1/login	IP-domain risk detected	Pass
4	Test suspicious keyword URL	URL containing verify-account-login	Suspicious keyword risk detected	Pass
5	Test shortened URL	Shortener-based URL	URL shortener warning displayed	Pass
6	Test AbuseIPDB lookup	URL domain resolved to IP	IP reputation checked successfully	Pass

Unit Testing 5: Malware File Scanner

Testing Objective: To ensure that file validation, hash generation, VirusTotal lookup, and live upload work correctly.

Table 6. 6: Malware File Scanner Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Upload supported file type	.pdf, .docx, .txt, or .zip	File accepted for scanning	Pass
2	Upload unsupported file type	Unsupported extension	Error message displayed	Pass
3	Upload large file	File exceeding size limit	File rejected safely	Pass
4	Generate SHA-256 hash	Valid uploaded file	Correct hash generated	Pass
5	VirusTotal hash lookup	Known file hash	VirusTotal report returned	Pass
6	Unknown file live upload	New file not found by hash lookup	File uploaded for live analysis	Pass
7	Poll VirusTotal analysis	Analysis ID returned	Final analysis result retrieved	Pass
8	Delete temporary file	File after scan	Temporary file removed	Pass

Unit Testing 6: Authentication, Database, and Scan History

Testing Objective: To ensure user accounts, sessions, and scan records work correctly.

Table 6. 7: Authentication and Database Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Register new user	Valid email and password	User record stored	Pass
2	Prevent duplicate registration	Existing email	Duplicate account error displayed	Pass
3	Login valid user	Correct credentials	User session starts	Pass
4	Login invalid user	Wrong credentials	Login rejected	Pass
5	Store email scan result	Completed email scan	Record saved in database	Pass
6	Store URL scan result	Completed URL scan	Record saved in database	Pass
7	Store file scan result	Completed file scan	Record saved in database	Pass
8	Retrieve scan history	Logged-in user	Previous scan records displayed	Pass

Unit Testing 7: Admin Panel and Email Notifications

Testing Objective: To ensure admin monitoring and notification delivery work correctly.

Table 6. 8: Admin Panel and Email Notification Unit Test Case

No.	Test Case / Test Script	Attribute and Value	Expected Result	Result
1	Login as admin	Admin credentials	Admin dashboard opens	Pass
2	View user list	Admin panel	Registered users displayed	Pass
3	View scan logs	Admin panel logs page	Logs displayed correctly	Pass
4	View threat statistics	Admin dashboard	Scan and threat statistics shown	Pass
5	Trigger email notification	Suspicious scan result	Email alert sent to user	Pass
6	Handle email failure	Invalid SMTP setting or unreachable service	Error logged without crashing system	Pass

6.1.3 Functional Testing

Functional testing was performed to verify that each functional requirement defined in Chapter 3 is implemented correctly. The purpose of functional testing is to ensure that every module performs its required operation from the user perspective.

Table 6. 9: Functional Testing

No.	Functional Requirement	Test Case / Test Script	Expected Result	Result
1	User registration	User creates new account	Account is created successfully	Pass
2	User login	User enters valid credentials	Dashboard opens	Pass
3	Email scan	User submits email text	Email verdict is displayed	Pass
4	Phishing classification	Submit phishing-style email	System identifies risk	Pass
5	Embedded link scanning	Submit email containing URL	Extracted link is scanned separately	Pass
6	Hidden .eml link detection	Upload .eml file with hidden href	Hidden link mismatch detected	Pass
7	URL analysis	User submits suspicious URL	URL risk result displayed	Pass
8	AbuseIPDB integration	Submit URL with resolvable domain	IP reputation is checked	Pass
9	File scanning	User uploads file	Malware scan result displayed	Pass
10	VirusTotal live upload	Upload unknown file	File is uploaded and analyzed	Pass

11	Result visualization	Complete any scan	Green/yellow/red result indicator shown	Pass
12	Scan history	User opens history page	Previous scans displayed	Pass
13	PDF report	User downloads report	Report downloaded successfully	Pass
14	Admin dashboard	Admin logs in	Admin statistics displayed	Pass
15	Email notification	Risky scan completed	Alert email sent	Pass
16	Gmail extension	Scan opened Gmail email	Extension shows verdict	Pass
17	Logout	User logs out	Session is cleared	Pass

Functional testing confirmed that all major user-facing features of CyberSentinel AI are working as expected.

6.1.4 Integration Testing

Integration testing was performed to verify communication between different modules. Since CyberSentinel AI includes frontend, backend, machine learning model, database, external APIs, email service, and browser extension, integration testing was important to ensure that all components work together correctly.

Table 6. 10: Integration Testing

No.	Integrated Modules	Test Case / Test Script	Expected Result	Result
1	React frontend + Flask backend	Submit email scan request	Backend receives request and returns result	Pass
2	Backend + phishing ML model	Send preprocessed email text to model	Model returns prediction and confidence	Pass
3	Email scan + URL analyzer	Submit email containing URL	Email and embedded link results are returned together	Pass
4	.eml parser + URL analyzer	Upload .eml with hidden link	Hidden link is extracted and scanned	Pass
5	URL analyzer + AbuseIPDB	Submit URL for scanning	IP reputation result included	Pass
6	File scanner + VirusTotal	Upload known file	VirusTotal report returned	Pass
7	File scanner + VirusTotal live upload	Upload unknown file	Live analysis result returned after polling	Pass
8	Backend + database	Complete any scan	Result stored in database	Pass
9	Frontend + scan history API	Open history page	Stored results displayed	Pass
10	Report generator + scan result	Generate PDF report	Report includes correct scan details	Pass
11	Scan module + email service	Suspicious result generated	Notification email sent	Pass

12	Admin panel + database	Admin opens logs/statistics	Data loaded from database	Pass
13	Gmail extension + live backend	Scan Gmail email from extension	Backend returns result to extension popup	Pass
14	Live frontend + live backend	Access from different network	Complete scan works successfully	Pass

Integration testing confirmed that the system modules communicate correctly and that the final deployed system works as a complete platform.

6.2 Automated Testing

Automated testing was performed to verify backend APIs, model predictions, input validation, and integration points more efficiently. Automated testing helped reduce repeated manual work and ensured that core endpoints responded correctly after changes.

Automated tests were mainly applied to backend endpoints, scanning modules, API responses, and selected frontend workflows. Tools such as Postman, browser developer tools, Python test scripts, SQLite Browser, Chrome extension developer mode, and deployment logs were used during testing.

6.2.1 Tools Used

Table 6. 11:Tools Used for Automated Testing

Tool Name	Tool Description	Applied On	Results
Postman	API testing tool used to send HTTP requests and inspect responses.	Login API, email scan API, URL scan API, file scan API, scan history API	APIs returned expected JSON responses
Python Test Scripts	Custom scripts used to test backend endpoints and prediction responses.	Phishing prediction, URL analyzer, malware scanner	Core backend functions tested successfully
Pytest / unittest	Python testing frameworks used for unit-level testing.	Input validation, text preprocessing, URL feature extraction, hash generation	Unit tests passed
Browser Developer Tools	Used to inspect frontend requests, console errors, and network responses.	React frontend and API integration	Frontend-backend communication verified
SQLite Browser	Tool used to inspect local database records.	Users, scan history, reports, logs	Records stored and retrieved correctly
Chrome Extension Developer Mode	Used to load and test the Gmail/Chrome extension.	manifest.json, content script, popup interface	Extension installed and scanned Gmail content
Render / Railway Logs	Deployment logs used to verify backend hosting behavior.	Live backend API	Backend deployed and reachable
Vercel / Netlify Logs	Deployment logs used to verify frontend hosting.	Live React frontend	Frontend deployed and connected to backend
Email Test Inbox	Used to verify delivery of system notifications.	Email notification module	Notification emails received successfully

6.2.2 Automated API Test Results

Table 6. 12: Automated API Test Results

No.	API / Module Tested	Test Input	Expected Response	Result
1	Login API	Valid email and password	Success response with session/token	Pass
2	Login API	Invalid password	Error response	Pass
3	Email Scan API	Phishing email text	Prediction, confidence, explanation	Pass
4	Email Scan API	Empty text	Validation error	Pass
5	Embedded Link API Logic	Email containing URL	Embedded link result list	Pass
6	.eml Upload API	Valid .eml file	Parsed email and hidden link results	Pass
7	URL Scan API	Suspicious URL	URL verdict and risk score	Pass
8	URL Scan API	Invalid URL	Validation error	Pass
9	File Scan API	Valid uploaded file	File scan result	Pass
10	File Scan API	Unsupported file	File type error	Pass
11	VirusTotal Upload Logic	Unknown file	Live analysis request created	Pass
12	Scan History API	Logged-in user request	User scan records returned	Pass
13	Admin API	Admin request	System statistics returned	Pass
14	Notification Service	Risky scan result	Email sent or logged	Pass
15	Extension Backend Request	Gmail email content	Prediction returned to extension	Pass

6.2.3 Performance Testing

Performance testing was conducted to check whether the system provides results within acceptable time. Since CyberSentinel AI depends on external APIs such as VirusTotal and AbuseIPDB, response time may vary depending on API availability and network conditions.

Table 6. 13: Performance Test Results

No.	Test Area	Expected Performance	Observed Result	Result
1	Email phishing prediction	Result within a few seconds	Result returned quickly	Pass
2	Embedded link extraction	Links extracted immediately	Links extracted successfully	Pass
3	URL analysis without API delay	Result within a few seconds	Result returned successfully	Pass

4	AbuseIPDB lookup	Result depends on API response	Result returned or graceful fallback applied	Pass
5	File hash generation	Hash generated quickly for allowed file size	Hash generated successfully	Pass
6	VirusTotal known file lookup	Result depends on API response	Result returned successfully	Pass
7	VirusTotal unknown file upload	Longer due to upload and polling	Result returned after polling	Pass
8	Scan history retrieval	Records load within acceptable time	History displayed successfully	Pass
9	Gmail extension scan	Result displayed in extension popup	Result displayed successfully	Pass

6.2.4 Security Testing

Security testing was performed to verify that the system includes basic protections against common web application issues. The system was tested for input validation, rate limiting, payload size control, API key protection, file handling safety, and role-based access.

Table 6. 14: Security Test Results

No.	Security Feature	Test Case / Test Script	Expected Result	Result
1	Input sanitization	Submit script tags in email input	Script removed or neutralized	Pass
2	Empty input validation	Submit blank scan request	Request rejected with error	Pass
3	File size limit	Upload file above allowed size	File rejected	Pass
4	File type validation	Upload unsupported file type	File rejected	Pass
5	API key protection	Inspect frontend code	API keys not visible in frontend	Pass
6	CORS configuration	Request from unauthorized origin	Request blocked or restricted	Pass
7	Password hashing	Inspect stored password	Plain password not stored	Pass
8	Role-based admin access	Normal user opens admin route	Access denied	Pass
9	Temporary file deletion	Scan uploaded file	Temporary file removed after processing	Pass
10	Rate limiting	Send repeated scan requests	Excessive requests limited	Pass

6.3 Summary

Testing and evaluation confirmed that CyberSentinel AI satisfies its major functional and non-functional requirements. Manual testing verified that the user interface, scanning modules, reports, notifications, admin panel, and extension work correctly. Unit testing confirmed that individual modules such as phishing detection, URL analysis, .eml parsing, malware scanning, authentication, and notifications function properly. Functional testing verified that all user-facing requirements are implemented, while integration testing confirmed successful communication between frontend, backend, ML models, database, external APIs, email service, and Gmail extension.

Automated testing further validated API responses, input handling, backend logic, and deployment behavior. The final system was tested as a deployed platform and confirmed to work from different devices and networks. Overall, the testing process shows that CyberSentinel AI is a reliable and functional cybersecurity toolkit suitable for final demonstration and evaluation.

7. Conclusion and Future Work

This chapter concludes the development and implementation of CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection. It summarizes the achievements of the project, highlights the effectiveness of the implemented modules, and discusses possible future enhancements that can further improve the system. The project successfully demonstrates how Artificial Intelligence, Machine Learning, Natural Language Processing, external threat intelligence APIs, secure web technologies, and user-centered design can be combined to build a practical cybersecurity solution for non-technical users.

CyberSentinel AI was developed to address the increasing threat of phishing emails, malicious URLs, and malware-infected files. The system provides an integrated platform where users can scan suspicious emails, links, and files in real time. It also includes scan history, report generation, admin monitoring, email notifications, Gmail extension support, hidden link detection in .eml files, and live deployment. These features make the system more complete, practical, and suitable for final-year project demonstration as well as real-world educational use.

7.1 Conclusion

CyberSentinel AI successfully achieves the main objective of providing a unified, intelligent, and user-friendly cybersecurity toolkit for detecting common digital threats. The system combines multiple detection techniques in a single platform instead of depending on only one method. This hybrid approach improves the system ability to analyze phishing emails, suspicious URLs, and potentially harmful files.

The phishing detection module uses Natural Language Processing and Machine Learning to classify email content as safe, suspicious, or phishing. Hybrid TF-IDF vectorization and Logistic Regression allow the system to identify phishing language, urgency-based wording, suspicious account-related messages, and obfuscated phishing patterns. The inclusion of Pakistani-specific phishing patterns improves the relevance of the system for local users who may receive scams related to JazzCash, Easypaisa, banks, NADRA, PTA, FBR, BISP, and similar services.

The URL threat analysis module strengthens the system by checking suspicious links using structural URL features and external reputation services. It analyzes indicators such as long URLs, suspicious keywords, IP-based domains, unsafe protocols, shorteners, unusual top-level domains, excessive subdomains, and long paths. Integration with AbuseIPDB and PhishTank further improves the reliability of URL analysis by adding live threat intelligence.

The malware file scanning module provides file-based threat detection using SHA-256 hashing and VirusTotal API integration. In the final implementation, the system was improved beyond simple hash lookup by adding live upload and polling for unknown files.

This makes the scanner more practical because a new file that is not already present in VirusTotal can still be submitted for live analysis.

The system also implements embedded link scanning inside emails and hidden link detection in .eml files. These features are important because phishing emails often hide malicious links behind safe-looking text. By extracting visible URLs and analyzing hidden HTML links, CyberSentinel AI provides deeper email inspection than a simple text classifier.

The final version further includes user authentication, persistent scan history, downloadable reports, admin panel, audit logging, email notifications, Gmail/Chrome extension, and live deployment. These modules improve usability, monitoring, accountability, and accessibility. The admin panel helps manage and monitor system activity, while email notifications alert users about suspicious or malicious scan results. The Gmail extension allows users to scan email content directly from Gmail, reducing the need for manual copy-paste.

Testing and evaluation confirmed that the system performs its required functions successfully. Manual testing, unit testing, functional testing, integration testing, automated API testing, security testing, deployment testing, and extension testing were performed to verify system behavior. The results show that CyberSentinel AI meets its functional and non-functional requirements and works as a complete cybersecurity toolkit.

Overall, CyberSentinel AI provides an accessible and practical cybersecurity solution for students, freelancers, small businesses, and general internet users. It simplifies threat detection by presenting results through clear verdicts, confidence scores, color-coded indicators, reports, and recommendations. The project demonstrates the successful application of AI, ML, NLP, web development, database systems, cybersecurity principles, and software engineering practices in solving a real-world problem.

7.2 Future Work

Although CyberSentinel AI has been completed as a functional and deployed cybersecurity toolkit, several future improvements can further enhance its accuracy, scalability, and real-world usability.

7.2.1 Larger Pakistani Cyber Threat Dataset

In future, the phishing dataset can be expanded with more real-world Pakistani phishing emails, scam messages, fake banking alerts, mobile wallet fraud examples, and malicious URLs. A larger local dataset would improve detection accuracy against threats targeting Pakistani users. The dataset can also include Urdu and Roman Urdu phishing messages to make the system more effective for local communication patterns.

7.2.2 Deep Learning-Based Phishing Detection

The current phishing detection model uses hybrid TF-IDF and Logistic Regression because it is fast, interpretable, and suitable for real-time classification. In future, advanced deep learning models such as BERT, DistilBERT, LSTM, or transformer-based classifiers can be

explored. These models may improve contextual understanding, especially for complex phishing emails that use indirect language or advanced social engineering.

7.2.3 Full Browser-Wide Extension

The current Gmail/Chrome extension focuses on scanning Gmail messages. In future, the extension can be expanded into a browser-wide security assistant that scans suspicious links, web pages, forms, and downloads across different websites. This would allow CyberSentinel AI to provide real-time protection beyond Gmail.

7.2.4 Mobile Application

A mobile application can be developed for Android and iOS users. Since many Pakistani users access digital banking, emails, and social media through mobile phones, a mobile version would make the system more accessible. The mobile app can include email scanning, URL scanning, file scanning, notifications, and report viewing.

7.2.5 Behavioral Malware Analysis

The current malware scanner uses VirusTotal and file analysis results. In future, behavioral malware analysis can be added using sandbox execution. This would allow the system to observe how a suspicious file behaves when executed in a controlled environment. Behavioral analysis can detect threats that may not be identified by static scanning alone.

7.2.6 Enterprise Dashboard and SIEM Integration

For organizational use, CyberSentinel AI can be extended with enterprise-level dashboards and integration with Security Information and Event Management systems. This would allow companies, universities, and small businesses to monitor threats, export logs, generate analytics, and integrate CyberSentinel AI with existing cybersecurity workflows.

7.2.7 Multi-Language Support

Future versions can support multiple languages, especially Urdu and Roman Urdu. This is important because many phishing messages in Pakistan use mixed English, Urdu, and Roman Urdu. Multi-language support would improve detection for local users and make the system more inclusive.

7.2.8 Two-Factor Authentication and Advanced Security

The authentication system can be improved by adding two-factor authentication, stronger password policies, session expiration controls, and advanced role-based permissions. These improvements would make the platform more secure for larger deployments.

7.2.9 Continuous Model Retraining

A feedback-based retraining pipeline can be added in future. If users or admins mark a scan result as correct or incorrect, the feedback can be stored and later used to improve the model. This would allow the system to adapt to new phishing patterns over time.

7.2.10 Cloud Scalability and Load Balancing

The deployed system can be improved through cloud scaling, load balancing, containerization, and monitoring tools. These improvements would help CyberSentinel AI handle more users and maintain stable performance under higher traffic.

7.3 Summary

CyberSentinel AI has been successfully developed as a complete, deployed, AI-assisted cybersecurity toolkit. It fulfills the project objectives by providing phishing detection, embedded and hidden link analysis, malicious URL scanning, malware file scanning, scan history, reports, admin panel, notifications, Gmail extension, and live deployment. Future improvements can make the system more advanced by adding larger datasets, deep learning models, mobile applications, behavioral malware analysis, browser-wide protection, enterprise integrations, and multi-language support.

8. References

The development of CyberSentinel AI - Intelligent Cybersecurity Toolkit for Phishing, Malware, and Threat Detection was supported by standard software engineering concepts, machine learning techniques, cybersecurity practices, official documentation, and external API references. The following sources were consulted during the design, development, implementation, testing, and documentation of the system.

1. R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, McGraw-Hill Education.

2. I. Sommerville, Software Engineering, Pearson Education.

3. C. D. Manning, P. Raghavan, and H. Schütze, Introduction to Information Retrieval, Cambridge University Press.

4. S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python, O'Reilly Media.

5. T. M. Mitchell, Machine Learning, McGraw-Hill Education.

6. C. M. Bishop, Pattern Recognition and Machine Learning, Springer.

7. D. Jurafsky and J. H. Martin, Speech and Language Processing, draft edition.

8. F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

9. Scikit-learn Developers, "Scikit-learn Documentation," https://scikit-learn.org/, Last accessed: July 2026.

10. Python Software Foundation, "Python Documentation," https://docs.python.org/, Last accessed: July 2026.

11. Pallets Projects, "Flask Documentation," https://flask.palletsprojects.com/, Last accessed: July 2026.

12. React Developers, "React Documentation," https://react.dev/, Last accessed: July 2026.

13. SQLite, "SQLite Documentation," https://www.sqlite.org/docs.html, Last accessed: July 2026.

14. VirusTotal, "VirusTotal API v3 Documentation," https://developers.virustotal.com/, Last accessed: July 2026.

15. AbuseIPDB, "AbuseIPDB API Documentation," https://www.abuseipdb.com/api.html, Last accessed: July 2026.

16. PhishTank, “PhishTank Developer Information,” https://phishtank.org/developer_info.php, Last accessed: July 2026.

17. OWASP Foundation, “OWASP Web Security Testing Guide,” https://owasp.org/www-project-web-security-testing-guide/, Last accessed: July 2026.

18. OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” https://owasp.org/www-project-top-ten/, Last accessed: July 2026.

19. NIST, “Cybersecurity Framework,” National Institute of Standards and Technology, https://www.nist.gov/cyberframework, Last accessed: July 2026.

20. IEEE, ISO/IEC/IEEE 29148: Systems and Software Engineering - Life Cycle Processes - Requirements Engineering, IEEE Standards Association.

21. Object Management Group, “Unified Modeling Language Specification,” https://www.uml.org/, Last accessed: July 2026.

22. GitHub Docs, “GitHub Documentation,” https://docs.github.com/, Last accessed: July 2026.

23. Postman, “Postman API Platform Documentation,” https://learning.postman.com/docs/, Last accessed: July 2026.

24. Google Chrome Developers, “Chrome Extensions Documentation,” https://developer.chrome.com/docs/extensions/, Last accessed: July 2026.

25. Mozilla Developer Network, “Web APIs and JavaScript Documentation,” https://developer.mozilla.org/, Last accessed: July 2026.

26. Flask-Limiter Documentation, “Rate Limiting for Flask Applications,” https://flask-limiter.readthedocs.io/, Last accessed: July 2026.

27. Python-dotenv Documentation, “Python-dotenv,” https://pypi.org/project/python-dotenv/, Last accessed: July 2026.

28. joblib Developers, “Joblib Documentation,” https://joblib.readthedocs.io/, Last accessed: July 2026.

29. SciPy Developers, “SciPy Documentation,” https://docs.scipy.org/doc/scipy/, Last accessed: July 2026.

30. jsPDF Developers, “jsPDF Documentation,” https://github.com/parallax/jsPDF, Last accessed: July 2026.

31. html2canvas Developers, “html2canvas Documentation,” https://html2canvas.hertzen.com/, Last accessed: July 2026.

32. Google, “Gmail Help and Gmail Security Information,” https://support.google.com/mail/, Last accessed: July 2026.

33. Render, “Render Documentation,” https://render.com/docs, Last accessed: July 2026.

34. Vercel, “Vercel Documentation,” https://vercel.com/docs, Last accessed: July 2026.

35. Netlify, “Netlify Documentation,” https://docs.netlify.com/, Last accessed: July 2026.

36. Kaggle, “Phishing Email Dataset Resources,” https://www.kaggle.com/, Last accessed: July 2026.

37. Cisco Talos Intelligence Group, “Talos Intelligence,” https://talosintelligence.com/, Last accessed: July 2026.

38. Kaspersky, “Threat Intelligence Portal,” https://opentip.kaspersky.com/, Last accessed: July 2026.

39. Google, “Google Safe Browsing,” https://safebrowsing.google.com/, Last accessed: July 2026.

40. Microsoft, “Microsoft Security Documentation,” https://learn.microsoft.com/en-us/security/, Last accessed: July 2026.